

A PROJECT REPORT ON

A Comprehensive Study and Implementation of Part-of-speech Tagging in the Assamese Language

*Submitted in partial fulfillment of the requirements for
the award of degree of*

Bachelor of Technology
in
Computer Science & Engineering



Submitted by

Aditya Ojha (200710007005)
Kushal Kachari (200710007030)
Rahul Das (200710007043)

Under the guidance of

Prof. Diganta Baishya

Associate Professor

(Department of Computer Science & Engineering, Jorhat Engineering College)

Department of Computer Science & Engineering
Jorhat Engineering College, Jorhat-785007, Assam
Assam Science and Technology University
(2023-2024)

Candidate Declaration

We hereby declare that the work which is being presented in this report entitled, “*A Comprehensive Study and Implementation of Part-of-speech Tagging in the Assamese Language*” submitted to the Department of Computer Science and Engineering of Jorhat Engineering College, is an authentic and original work done under the guidance of Prof. Diganta Baishya, Associate Professor, CSE, Jorhat Engineering College and is submitted in fulfilment of the requirements for the Degree of Bachelor of Technology in Computer Science and Engineering. The result embodied in this report has not been submitted elsewhere for any credit or requirement.

Date:

Place:

Aditya Ojha

200710007005

Kushal Kachari

200710007030

Rahul Das

200710007043

Certificate from Institution

This is to certify that the project entitled, "*A Comprehensive Study and Implementation of Part-of-speech Tagging in the Assamese Language*" is a bonafide work carried out by Aditya Ojha (Roll no. 200710007005), Kushal Kachari (Roll no. 200710007030) and Rahul Das (Roll no. 200710007043) under my guidance and supervision, for the fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Jorhat Engineering College.

Date:

Place:

Dr. Rupam Baruah

Head of Department

Dept. of Computer Science &
Engineering

Jorhat Engineering College

Jorhat-785007, Assam

Certificate from Project Guide

This is to certify that the project entitled, "*A Comprehensive Study and Implementation of Part-of-speech Tagging in the Assamese Language*" is a bonafide work carried out by Aditya Ojha (Roll no. 200710007005), Kushal Kachari (Roll no. 200710007030) and Rahul Das (Roll no. 200710007043) under my guidance and supervision, for the fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Jorhat Engineering College.

The project work has been carried out during the academic year 2023-24 under my supervision and has not been submitted elsewhere for any purpose. The work presented in this project report is original and has not been copied from any source.

Date:

Place:

Prof. Diganta Baishya

Associate Professor

Dept. of Computer Science &
Engineering

Jorhat Engineering College

Jorhat-785007, Assam

Certificate of Examination

This is to certify that Aditya Ojha (Roll no. 200710007005), Kushal Kachari (Roll no. 200710007030) and Rahul Das (Roll no. 200710007043) of B.Tech 8th semester of the department of Computer Science and Engineering from Jorhat Engineering College have completed the project entitled, "*A Comprehensive Study and Implementation of Part-of-speech Tagging in the Assamese Language*" under the guidance of Prof. Diganta Baishya, Associate Professor, Department of Computer Science and Engineering, Jorhat Engineering College. We have examined this report and hereby record and approve for it as a successful project work.

External Examiner

Date:

Place:

Acknowledgement

We would like to express our heartfelt gratitude to all the people who inspired, supported, helped us in completion of our project. We are very grateful to our Head of the Department, Dr. Rupam Baruah for providing such an opportunity to take up research on an industrially inclined topic which will in turn broaden our field of interests. We are deeply indebted to our project guide Prof. Diganta Baishya for his excellent ideas, advice, support and thorough guidance during our project.

At last but certainly not least, we express our gratitude to our professors, staff and classmates of the Department of Computer Science and Engineering for rendering their cooperation, and everyone who has directly or indirectly helped us in completing our project.

Date:

Place:

Aditya Ojha

200710007005

Kushal Kachari

200710007030

Rahul Das

200710007043

Abstract

This project works on conducting an extensive study and implementation of part-of-speech (POS) tagging in the Assamese language, a significant yet underrepresented Indo-Aryan language spoken in northeastern India. The primary objective of this study is to understand, implement and analyze the results that were concluded in the reference paper. Moreover, the aim is also to leverage those results and further work on refining the studies conducted in the field. By leveraging both traditional and state-of-the-art deep learning techniques, this study aims to enhance the accuracy and efficiency of POS tagging for Assamese. The project acts as an adequate implementation of studies done earlier in this field. To thoroughly evaluate the effectiveness of various approaches, the study employs different levels of text representation, including word-level, character-level, bigram, and trigram models. The experimental setup involves training these models on both small and large datasets, followed by a rigorous evaluation of their performance using standard accuracy metrics. The comparative analysis focuses on identifying the strengths and limitations of each method, with particular attention to the implemented activation function's impact on model accuracy and robustness. The results demonstrate that the custom PSM-Taylor SoftMax activation function significantly enhances the performance of deep learning models, especially in scenarios with limited training data. The combination of probabilistic and deep learning approaches outperforms these existing standalone approaches in terms of capturing complex linguistic patterns and achieving higher tagging accuracy. The findings underscore the potential of advanced deep learning techniques in overcoming the resource limitations inherent to low-resource languages.

Contents

Candidate Declaration	i
Certificate from Institution	ii
Certificate from Project Guide	iii
Certificate of Examination	iv
Acknowledgement	v
Abstract	vi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Project Objectives	2
1.3 Key Terminologies	3
1.4 Python for Deep Learning	5
1.4.1 TensorFlow	6
1.4.2 Keras	6
1.5 Organization of Chapters	6
2 Literature Review	8
2.1 Introduction	8
2.2 Related Works	9
2.3 Methodology	13
2.4 Experimental Results and Observations	13
2.5 Conclusion	14
3 Methodology	15

3.1	Tags	15
3.2	Dataset	16
3.2.1	Dataset 1: Small	16
3.2.2	Dataset 2: Large	17
3.3	Theoretical Concepts	17
3.4	Structure of the Project	18
3.4.1	Layer 4: Dataset Size	18
3.4.2	Layer 3: Approach	19
3.4.3	Layer 2: Method of Tagging	20
3.4.4	Layer 1: Alteration in Functions	24
3.5	Flow of Code	25
3.5.1	Importing Libraries	25
3.5.2	Importing Data	25
3.5.3	Defining Classes and Functions	26
3.5.4	Data Preprocessing	27
3.5.5	Model Architecture	28
3.5.6	Training and Evaluation	28
4	Results and Observations	30
4.1	Results under Dataset 1: Small	30
4.1.1	Standard Approach	30
4.1.2	Probabilistic Approach	34
4.1.3	Comparison of Standard and Probabilistic Approaches . . .	39
4.2	Results under Dataset 2: Large	40
4.2.1	Standard Approach	40
4.2.2	Probabilistic Approach	44
4.2.3	Comparison of Standard and Probabilistic Approaches . . .	48
4.3	Observations	49
4.3.1	Dataset 1 (Small)	49
4.3.2	Dataset 2 (Large)	49
4.3.3	General Observations	50
5	Application of Results	52
5.1	About	53

5.1.1	Flask	53
5.1.2	Render	53
5.2	Development of Application	54
5.2.1	Libraries and Setup	54
5.2.2	Flask App Initialization	54
5.2.3	Data Loading and Processing	54
5.2.4	Model Configuration and Loading	55
5.2.5	Flask Route for Handling Requests	56
5.2.6	HTML Template	57
5.3	Deployment of Application	57
5.3.1	Preparing the Flask App for Deployment	57
5.3.2	Pushing Code to GitHub	57
5.3.3	Deploying on Render	58
5.3.4	Accessing the App	58
5.4	Working of Application	58
6	Future Prospects	60
7	Conclusion	61
	References	62

Chapter 1

Introduction

POS tagging is a well known Natural Language Processing (NLP) problem of automatically assigning grammatical categories to words in a text. Recent breakthroughs, particularly in deep learning, have propelled it forward. However, the efficacy of these methods often relies on vast amounts of training data. As a result, low resource languages like Assamese remains unexplored. This project aims to resolve this drawback by hovering over different approaches. It implements 'PSM Taylor Softmax function' and a custom loss function from scratch, treats a word in various forms viz. character, bigram, trigram etc. for model training, and also takes in account probability distribution of the words. The models are then trained for both small and large datasets, comparisons are carried out among the obtained results and the intricacies are analyzed.

1.1 Background and Motivation

Natural language processing (NLP) is a pivotal field in artificial intelligence that focuses on the interaction between computers and human language. POS tagging is a critical task within NLP that assigns parts of speech to each word in a sentence, forming the foundation for numerous downstream applications such as syntactic parsing, machine translation, and information retrieval. While significant progress

has been made in POS tagging for high-resource languages, low-resource languages still face substantial hurdles due to limited annotated data and linguistic resources.

The paper *"Part-of-Speech Tagging for Low-Resource Languages: Activation Function for Deep Learning Network to Work with Minimal Training Data"*[1] proposed multiple ways of dealing with, training and testing Assamese data for POS tagging and then proposed a custom activation function for the same. This project aims to not only implement the results concluded in that paper, but also act as an extension to different other approaches for POS tagging.

Traditional POS tagging methods, including rule-based approaches and probabilistic models such as Naive Bayes classifiers, Hidden Markov Models (HMMs), and Conditional Random Fields (CRFs), have shown varying degrees of success. More recently, deep learning models like Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), Gated Recurrent Units (GRUs), and Bidirectional LSTMs (Bi-LSTMs) have achieved remarkable results by leveraging large datasets to capture complex linguistic patterns. However, these deep learning techniques generally require extensive training data, which is often unavailable for low-resource languages.

1.2 Project Objectives

While significant progress has been made in POS tagging for well-resourced languages like English, low-resource languages often struggle due to a lack of annotated data and linguistic resources. The study specifically focuses on Assamese, which represents the challenges encountered by low-resource languages. The main purpose of this study is to implementing and extending the ideas and findings that were concluded in [1] and then develop a robust and efficient POS tagging tools that can be effectively utilized for low-resource languages. This research not only addresses the specific challenges faced by Assamese but also offers methodologies that can be applied to other low-resource languages, thereby contributing to the broader effort of making advanced NLP techniques accessible and effective for all languages.

1.3 Key Terminologies

1. **Part-of-Speech (POS) Tagging:** Part-of-Speech tagging, often abbreviated as POS tagging, is a process in natural language processing (NLP) where each word in a sentence is assigned a part of speech, such as noun, verb, adjective, etc. This task is fundamental for understanding the grammatical structure of sentences and is a prerequisite for various other NLP applications like parsing, machine translation, and information retrieval.
2. **Natural Language Processing (NLP):** Natural Language Processing is a field of artificial intelligence that focuses on the interaction between computers and human languages. It involves the development of algorithms and models that enable computers to understand, interpret, and generate human language. Applications of NLP include speech recognition, sentiment analysis, language translation, and chatbots.
3. **Low-Resource Languages:** Low-resource languages are languages that lack substantial digital resources, such as large annotated corpora, comprehensive dictionaries, and language models. This scarcity of resources poses significant challenges for developing NLP tools and techniques that perform well for these languages. Assamese is an example of a low-resource language.
4. **Conditional Random Fields (CRFs):** Conditional Random Fields are a type of probabilistic graphical model used for sequence prediction. CRFs are particularly effective for labeling and segmenting sequential data, making them useful for tasks like POS tagging. They model the conditional probability of the output sequence given the input sequence, considering the dependencies between output labels.
5. **Recurrent Neural Networks (RNNs):** Recurrent Neural Networks are a class of neural networks designed to recognize patterns in sequences of data, such as time series or natural language. RNNs have connections that form directed cycles, allowing them to maintain a form of memory across sequences. This makes them suitable for tasks like language modeling and POS tagging.

6. **Long Short-Term Memory Networks (LSTMs):** Long Short-Term Memory networks are a type of RNN that addresses the problem of learning long-term dependencies. LSTMs incorporate memory cells that can maintain information over long periods, which helps in capturing context in sequential data. This makes LSTMs highly effective for tasks involving long-range dependencies, such as POS tagging and language translation.
7. **Gated Recurrent Units (GRUs):** Gated Recurrent Units are a variant of RNNs that aim to simplify the architecture of LSTMs while retaining their ability to model long-term dependencies. GRUs use gating mechanisms to control the flow of information, making them computationally efficient and effective for sequence-based tasks.
8. **Bidirectional LSTMs (Bi-LSTMs):** Bidirectional Long Short-Term Memory networks extend the standard LSTM architecture by processing the input sequence in both forward and backward directions. This allows Bi-LSTMs to capture context from both past and future states, improving performance on tasks where understanding the full context is crucial, such as POS tagging.
9. **Activation Functions:** Activation functions are mathematical functions used in neural networks to introduce non-linearity into the model. Common activation functions include ReLU (Rectified Linear Unit), sigmoid, tanh, and SoftMax. The choice of activation function can significantly affect the performance and learning dynamics of the model.
10. **PSM-Taylor SoftMax:** PSM-Taylor SoftMax is an activation function implemented in this project as per the findings in [1]. It combines the statistical probability distribution of words over tags with the SMTaylor SoftMax function, aiming to improve the accuracy of POS tagging, especially for low-resource languages. This activation function is designed to enhance the model's ability to generalize from limited training data.
11. **Annotated Corpora:** Annotated corpora are collections of text data that have been labeled with various linguistic annotations, such as parts of speech, syntactic structures, or named entities. These corpora are essential for train-

ing and evaluating NLP models, providing the ground truth needed for supervised learning algorithms.

12. **Bigram:** A bigram is a sequence of two adjacent elements from a string of tokens, which can be letters, syllables, or words. In the context of natural language processing, bigrams are typically used to analyze the relationship between two consecutive words in a text. By examining word pairs, bigram models capture local dependencies and context within the text, which helps in understanding the structure and semantics of the language. Bigrams are especially useful in tasks such as language modeling, text classification, and machine translation, where the immediate context plays a crucial role in determining the meaning and flow of the text.
13. **Trigram:** A trigram is a sequence of three adjacent elements from a string of tokens, which can be letters, syllables, or words. In natural language processing, trigrams extend the concept of bigrams by including an additional word, thus capturing slightly longer dependencies and more context within the text. Trigram models are valuable for capturing the syntactic and semantic relationships between words, providing a richer representation of the text compared to bigrams. This makes trigrams particularly useful in applications such as language modeling, speech recognition, and predictive text input, where understanding a wider context improves accuracy and performance.

1.4 Python for Deep Learning

Presently, the most sought-for programming language in the machine learning professional field is Python. Python's popularity may be due to the increased development of deep learning frameworks available for this language in the recent times. Python is an interpreted high-level general-purpose programming language. Python's design philosophy emphasizes code readability with its notable use of significant indentation. Its language constructs as well as its object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

Python offers concise and readable code. While complex algorithms and versatile workflows stand behind machine learning and AI, Python's simplicity allows developers to write reliable systems. It is considered by many as one of the easiest programming language, if not, the easiest programming language. Python programming language provides us with a variety of reasons for its stand as one of the most used programming languages. To reduce development time Python provides us with various frameworks and libraries. Libraries are pre-written codes to solve common programming tasks. Python provides us with several python frameworks and libraries to make our work easier and enable us to complete our work faster. There are multiple python libraries utilized in this project. However the most important of them are:

1.4.1 TensorFlow

TensorFlow is an open-source machine learning library developed by the Google Brain team. It is designed to facilitate the creation, training, and deployment of machine learning models, particularly deep neural networks. TensorFlow is widely used in both research and production environments due to its flexibility, scalability, and comprehensive ecosystem of tools and libraries.

1.4.2 Keras

Keras is an open-source neural network library written in Python. It is designed to enable fast experimentation with deep neural networks and is user-friendly, modular, and extensible. Initially developed as a part of the research effort of the ONEIROS (Open-ended Neuro-Electronic Intelligent Robot Operating System) project, Keras has since become an integral part of the TensorFlow ecosystem. Keras is integrated into the TensorFlow library.

1.5 Organization of Chapters

This project report is structured to systematically present the research, methodologies, findings, and conclusions related to the enhancement of part-of-speech (POS)

tagging for low-resource languages, with a focus on Assamese. The organization of chapters is as follows:

- **Introduction**
- **Literature Review**
- **Methodology**
- **Results and Observations**
- **Application of Results**
- **Future Prospects**
- **Conclusion**

Chapter 2

Literature Review

This literature review delves into the methodologies, experimental results, and conclusions of the research paper titled "*Part-of-Speech Tagging for Low-Resource Languages: Activation Function for Deep Learning Network to Work with Minimal Training Data*" by Diganta Baishya and Rupam Baruah (2024). The study explores the efficacy of various machine learning techniques, particularly focusing on deep learning models, for part-of-speech (POS) tagging in languages with limited training data, specifically English and Assamese. The paper compares the performance of Conditional Random Fields (CRF) with modern deep learning architectures like RNN, LSTM, GRU, and Bi-LSTM, analyzing the impact of different activation functions and the size of training datasets.

2.1 Introduction

Part-of-speech tagging is a fundamental task in natural language processing (NLP) that assigns parts of speech to each word in a sentence, playing a crucial role in various NLP applications such as syntactic parsing, machine translation, and information retrieval. This task becomes particularly challenging for low-resource languages due to the scarcity of annotated training data, which hampers the development of accurate and reliable tagging models. The paper under review seeks

to address this challenge by conducting a comprehensive evaluation of both traditional and advanced methods for POS tagging, specifically focusing on English and Assamese languages. Traditional methods, such as Conditional Random Fields (CRF), are assessed alongside advanced deep learning models, including Recurrent Neural Networks (RNN), Long Short-Term Memory networks (LSTM), Gated Recurrent Units (GRU), and Bidirectional LSTM (Bi-LSTM). These models are analyzed for their performance with varying sizes of training data and different activation functions. The study not only investigates how these models perform under different conditions but also implements a very efficient "PSM-Taylor Soft-Max" activation function, designed to enhance tagging accuracy in scenarios with minimal training data; as per the findings in [1]. This new activation function is evaluated for its potential to improve the performance of POS tagging models, particularly in low-resource settings where data limitations are a significant barrier.

2.2 Related Works

The literature review of related works provides a comprehensive overview of previous research in POS tagging. Key studies include:

1. **Martinez (2012)** Described state-of-the-art methods for Part-of-Speech (POS) tagging, highlighting the popularity of both stochastic methods and rule-based methods.
2. **Màrquez et al. (2000)** Resolved POS ambiguity using supervised learning and machine learning techniques. Utilized numerous statistical decision trees to represent the distribution of tags and words in various contexts.
3. **Liang Lu (2003)** Implemented a POS tagger using a min-max modular neural network model.
4. **Santos et al. (2014)** Implemented POS tagging using a deep neural network that learns character-level representations of words and integrates them with conventional word representations.

5. **Chiche and Yitagesu (2022)** Conducted a comprehensive review of the latest articles on POS tagging methodologies, focusing particularly on machine learning and deep learning approaches.
6. **Socher et al. (2012)** Concluded that recursive neural networks have the capability to learn structured tree outputs and vector representations of phrases and sentences. Additionally, demonstrated the efficacy of a modified back-propagation method in achieving improved training outcomes.
7. **Tanya Dayanand (2020)** Demonstrated the implementation of several popular deep learning models, evaluating their strengths and weaknesses. Achieved significant accuracy through extensive training dataset utilization within the study.
8. **Fahad et al. (2018)** Conducted a systematic review of tools and techniques that illustrate the relationship between Natural Language Processing (NLP) and Deep Learning.
9. **Horsmann et al. (2017) and Plank et al. (2016)** Reported that LSTM-based POS taggers considerably improve POS tagging.
10. **Balwant (2019)** Presented a CNN-based architecture to incorporate speaker profiles and parts of speech tags from news articles.
11. **Xiaorui and Zhang (2021)** Proposed a new POS tagger that complies with construction codes. Integrated error-driven transformational rules and deep learning neural network models.
12. **Zheng et al. (2013), Baishya et al. (2021), Baishya et al. (2021), Deshmukh et al. (2020)** Contributed towards the use of deep learning in Indian and other South Asian languages.
13. **Lafferty et al. (2001)** Presented a framework of conditional random fields (CRF) to build probabilistic models for segmenting and labeling sequence data.
14. **Cohn (2007)** Reported two significant flaws of CRFs: frequent overfitting and slow standard training methods.

15. **CA Sutton (2008)** Investigated efficient training methods for CRFs with complex graphical structures, focusing on local methods.
16. **Huang et al. (2015)** Implemented mixing deep learning with CRF, working on various neural network architectures such as LSTM, Bi-LSTM, and CRF layers.
17. **Chen et al. (2020)** Presented a multi-channel architecture dividing statements into multiple channels containing stacked neural networks.
18. **W. Khan et al. (2019)** Developed deep learning models for Urdu POS tagging, demonstrating the effectiveness of machine learning and deep learning approaches with a well-balanced feature set.
19. **Kim et al. (2019)** Introduced a method for automatically generating machine-labeled biomedical NER corpora from a unified medical language system (UMLS). They employed a bootstrapping approach with a small initial corpus and trained CRF classifiers and LSTM models to enhance performance.
20. **Sornlertlamvanich and Yuenyong (2022)** Identified common issues in the morphological analysis of Asian languages. They emphasized the significant impact of POS tagging on subsequent text processing tasks and proposed enhancements to Named Entity Recognition using Thai character cluster (TCC) integration into Bi-LSTM models.
21. **AlKhawter and Al-Twairsh (2021)** Utilized Conditional Random Fields and Bidirectional Long Short-Term Memory (BiLSTM) models for POS tagging. They presented datasets for Arabic tweet tagging and examined hashtag usage behavior. Their findings highlighted the superiority of Bi-LSTM-based POS tagging for specific datasets.
22. **Zeroual et al. (2017)** Developed a hierarchical categorization of Arabic tagsets, facilitating expansion and yielding accurate outcomes. They implemented this tagset in a POS tagger and conducted extensive evaluations.

23. **Nguyen et al. (2018)** Proposed a Bi-LSTM-CRF model variant to improve deep learning performance without extensive annotated data. They addressed overlapping RE recognition challenges and demonstrated the model's effectiveness on Japanese law datasets.
24. **Kumar et al. (2020)** Demonstrated a combined Bi-LSTM and CNN model for POS tagging, leveraging temporal data retention and feature extraction. Their model achieved improved performance with reduced execution time.
25. **Besharati et al. (2021)** Explored Persian POS tagging and proposed hybrid models integrating HMM with bidirectional LSTM. They highlighted the superior performance of neural models in predicting POS tags for out-of-vocabulary words.
26. **Bhattu et al. (2020)** Presented methods for constructing word embeddings in code-mixed text scenarios. They enhanced POS tagging effectiveness in mixed-script text using hierarchical deep recurrent neural networks with CRF layers.
27. **Warjri et al. (2021)** Employed deep learning techniques on a self-developed Khasi POS corpus. They introduced Bi-LSTM-based POS taggers and character-based embedding approaches.
28. **Banerjee et al. (2021)** Investigated alternatives to SoftMax, including log-Taylor SoftMax loss and SM-SoftMax. They introduced SM-Taylor SoftMax, a hybrid of Taylor SoftMax and SM-SoftMax, which outperformed traditional SoftMax functions.
29. **Shi et al. (2019)** Implemented cascaded SoftMax to develop a fine-grained image classifier.
30. **Luo et al. (2020)** Proposed a Gaussian-based SoftMax (G-SoftMax) function to enhance intraclass compactness and interclass separability.
31. **Wang et al. (2018)** Introduced an additive margin SoftMax mechanism for Face Verification.

32. **Liu et al. (2016) and Liang et al. (2017)** Described approaches for large margin SoftMax and soft margin SoftMax. They proposed L-SoftMax, a generalized large-margin SoftMax loss, improving intra-class compactness and inter-class separability. Additionally, they presented a method for large margin SoftMax loss for CNN, leading to more robust classification.

2.3 Methodology

The research employs a comparative approach to evaluate the performance of CRF and several deep learning models under constraints of limited training data. The methodology includes:

- **Dataset Preprocessing:** Creating training and testing datasets for both English and Assamese languages with varying sizes.
- **Model Training:** Implementing CRF and deep learning models (RNN, LSTM, GRU, Bi-LSTM) using different activation functions such as ReLU, Sigmoid, Tanh, and SoftMax.
- **Evaluation Metrics:** Assessing model performance based on accuracy, analyzing the effect of the number of epochs, and comparing the performance across different corpus sizes.
- **Proposed Activation Function:** Implementing a very efficient activation function proposed in [1], "PSM-Taylor Softmax", aimed at improving POS tagging accuracy for low-resource languages.

2.4 Experimental Results and Observations

1. **CRF Performance:** CRF shows reasonable accuracy with large training datasets but struggles with minimal training data, indicating its limitations for low-resource languages.
2. **Deep Learning Models:** Among the deep learning models, LSTM and Bi-LSTM outperform basic RNNs, demonstrating significant improvements

even with limited training data. The accuracy of these models increases with larger datasets and more epochs, though a threshold is reached beyond which no further improvements are observed.

3. **Activation Functions:** The SoftMax activation function consistently outperforms others in the context of POS tagging for both languages, making it particularly effective for multiclass classification tasks. The Sigmoid function also performs comparably well.
4. **Language Comparison:** The models exhibit better performance with Assamese than with English, suggesting that the models learn more effectively with the Assamese dataset despite similar training sizes.
5. **Proposed Activation Function:** The implemented "PSM-Taylor Soft-max" function, which is referred from [1], shows promise in enhancing accuracy, especially for unseen words and insufficient training data, suggesting its potential utility for various NLP applications involving low-resource languages.

2.5 Conclusion

The research concludes that CRFs, while effective with large training datasets, are not suitable for languages with limited resources. Advanced deep learning models, particularly those using the SoftMax activation function, show promising results even with minimal training data. The proposed PSM-Taylor SoftMax activation function further enhances model performance, making it a valuable tool for developing NLP applications for low-resource languages. The findings have significant implications for the design of POS taggers and other NLP tools for languages lacking extensive digital text corpora.

Chapter 3

Methodology

As stated in Chapter 1, the project serves as a real-world implementation of the paper "*Part-of-Speech Tagging for Low-Resource Languages: Activation Function for Deep Learning Network to Work with Minimal Training Data*".

The paper primarily concludes that for POS-tagging, using Bi-LSTM with SMTaylor SoftMax functions gave the most accurate results. This project further extends the experiments and observations of that paper and goes deeper into different alterations in various versions of the model in order to obtain the most optimal results. Moreover, this project also aims at introducing custom activation and loss functions and comparing them with the already existing activation functions in the TensorFlow library.

The following sections explain the overall methodology that has been followed for this project.

3.1 Tags

There are many POS tags available in multiple languages. By following the work done in [1], the following tags have been considered for the project:

1. **N:** NOUN
2. **Pr:** PRONOUN
3. **Adj:** ADJECTIVE
4. **V:** VERB
5. **ADV:** ADVERB
6. **PREP:** PREPOSITION
7. **CON:** CONJUNCTION
8. **INT:** INTERJECTION

3.2 Dataset

A total of two datasets have been utilized for this project: a small and a large one. This is done so as to test the capabilities of the models in narrow as well as more diverse data. The tagging in the datasets were initially made as a stream of words. However, for this project, sentence numbers have also been given priority. The structure of both the datasets can be viewed in Table 3.1.

Table 3.1: Structure of Dataset

Sentence #	Word	POS Tag
-	-	-

It is also worth noting that the datasets already contain the tags corresponding to each word, hence this project is based on a supervised methodology of POS-tagging.

3.2.1 Dataset 1: Small

This dataset is a combination of corpora from multiple resources in the Assamese language. It has a total of **448 sentences** and **5,192 words**.

3.2.2 Dataset 2: Large

This dataset too is a combination of corpora from multiple resources in the Asamese language. However, it is significantly larger than Dataset 1. It has a total of **11,764 sentences** and **1,70,209 words**.

3.3 Theoretical Concepts

Chapter 1 and 2 gave an overview of the studies and implementations that are done in this project. This section of Chapter 3 is focused on the exact changes that are made in generic *Softmax* activation function and *Categorical Crossentropy* loss function.

A generic *Softmax* activation function is denoted as shown in Equation (3.1).

$$Softmax(z)_i = \frac{e^{z_i}}{\sum_j^k e^{z_j}} \quad \text{for } i, j = 1, \dots, K \text{ and } z = (z_1, \dots, z_k) \in R^k. \quad (3.1)$$

The proposed modified activation function is denoted as shown in Equation (3.2).

$$CustomSoftmax(z)_i = \frac{e^{z_i - \epsilon}}{\sum_j^k e^{z_j - \epsilon}} \quad \text{for } i, j = 1, \dots, K \text{ and } z = (z_1, \dots, z_k) \in R^k. \quad (3.2)$$

A generic *Categorical Crossentropy* loss function is denoted as shown in Equation (3.3).

$$CatCrossLoss(y_{\text{true}}, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{\text{true}, i, j} \cdot \log(\hat{y}_{i, j}) \quad (3.3)$$

N : Number of samples in the batch,
 C : Number of classes,
 y_{true} : True label (one-hot encoded),
 $\hat{y}$: Predicted probability distribution over classes.

The proposed modified loss function in this project can be viewed in Equation (3.4).

$$\text{CustomCatCrossLoss}(y_{\text{true}}, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N (y_{\text{true},i} \cdot \log(\text{clip}(\hat{y}_i, \epsilon, 1 - \epsilon))) \quad (3.4)$$

N : Number of samples in the batch,
 C : Number of classes,
 y_{true} : True label (one-hot encoded),
 $\hat{y}$: Predicted probability distribution over classes.

3.4 Structure of the Project

The structure of the project can be understood in a layer-wise manner, with layer 4 being the outermost layer and layer 1 being the innermost layer.

Figure 3.1 gives an overview of the structure of the project. In-depth understanding on each component of the project's structure can be obtained by going through the sections that follow.

3.4.1 Layer 4: Dataset Size

In the outermost layer, there are the two datasets (as mentioned in section 3.2) on which the various experiments have been conducted.

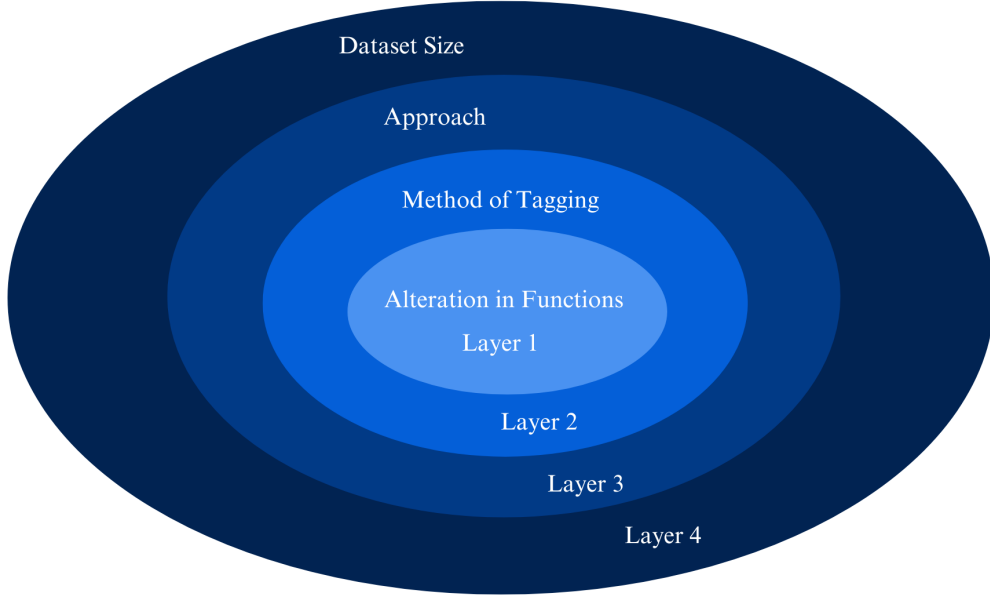


Figure 3.1: *Structure of the Project*

3.4.2 Layer 3: Approach

Under each dataset, two approaches have been followed for the experimentation. Both of them use Bidirectional LSTM as the model architecture as this is the one which performed the best in the experimentation of [1].

1. **Standard Approach:** Here, a generic method of training and testing has been followed. The models have been trained as per the specifications normally followed for training Bi-LSTM Neural Networks.

The activation function gives a probabilistic distribution of each of the words with their corresponding tags. Each word is represented as an array of 7 tags, with each of them denoting the probability of that word being assigned to that tag. The tag with the highest probability is assigned to the concerned word.

2. **Probabilistic Approach:** Here, a similar approach is followed as in the standard approach. Additionally, an external probability distribution of each

of the word is created by considering the words' occurrences in the dataset. This probability distribution array is then multiplied (dot product) with the array of tags obtained in the standard approach. Ultimately, in the resultant array, the tag with the highest probability is assigned to the concerned word.

3.4.3 Layer 2: Method of Tagging

Under each approach, the following methods have been employed for doing the task of tagging differently:

1. **Word Tagging:** This is the most generic method followed. In the dataset, each word is assigned with a tag. Here, the model is trained on the basis of the word-tag pairs. While training the same mapping is taken into consideration. The method of word tagging can be seen in Figure 3.2.

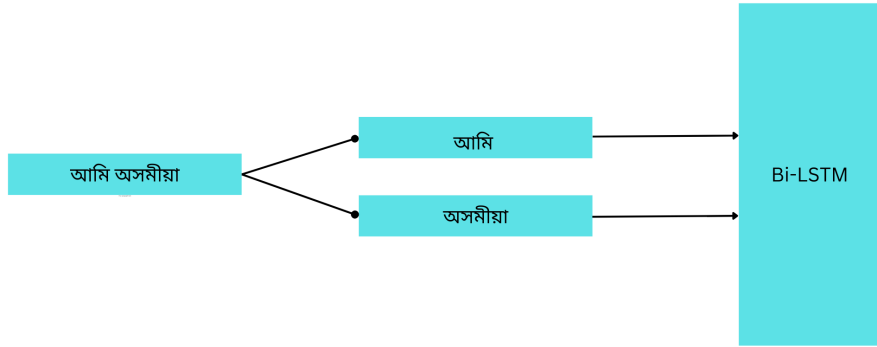


Figure 3.2: *Word Tagging*

2. **Character Tagging:** Here, each word is split into its corresponding characters. Then, the characters are assigned with the corresponding tags of their parent words. Here, the model is trained on the basis of the character-tag pairs. While training the same mapping is taken into consideration. The method of character tagging can be seen in Figure 3.3.

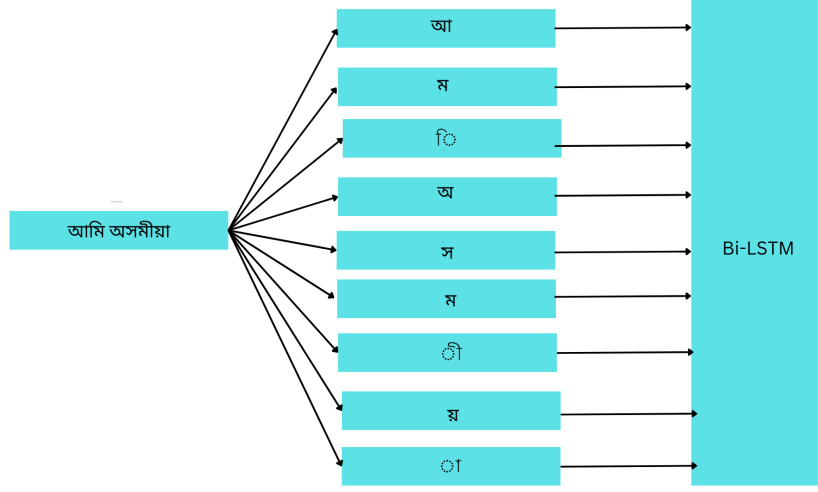


Figure 3.3: *Character Tagging*

3. **Bigram Tagging:** Here, each word is split into its corresponding bigrams. Then, the bigrams are assigned with the corresponding tags of their parent words. Here, the model is trained on the basis of the bigram-tag pairs. While training the same mapping is taken into consideration. The method of bigram tagging can be seen in Figure 3.4.

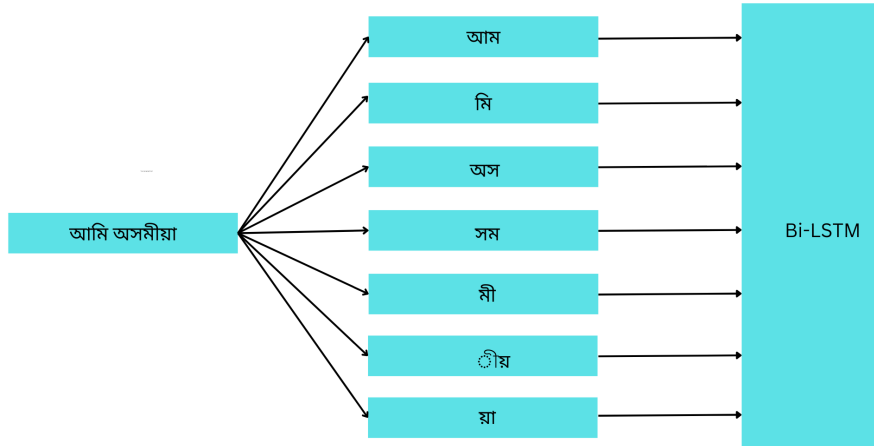


Figure 3.4: *Bigram Tagging*

4. **Trigram Tagging:** Here, each word is split into its corresponding trigrams.

Then, the trigrams are assigned with the corresponding tags of their parent words. Here, the model is trained on the basis of the trigram-tag pairs. While training the same mapping is taken into consideration. The method of trigram tagging can be seen in Figure 3.5.

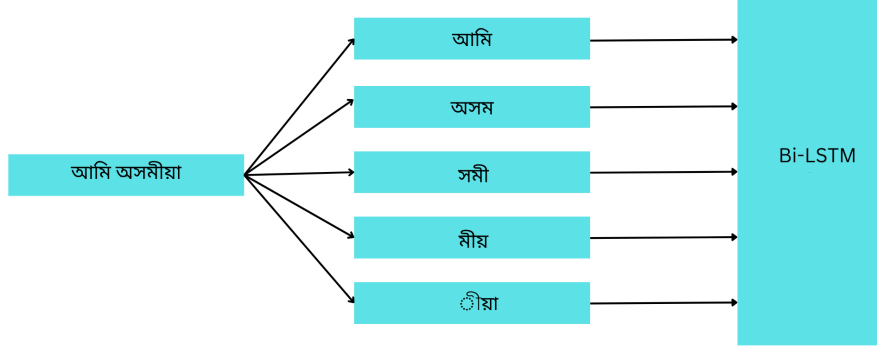


Figure 3.5: *Trigram Tagging*

5. **Word-Character Tagging:** It is a combination of Word Tagging and Character Tagging, done simultaneously. The method of word-character tagging can be seen in Figure 3.6.

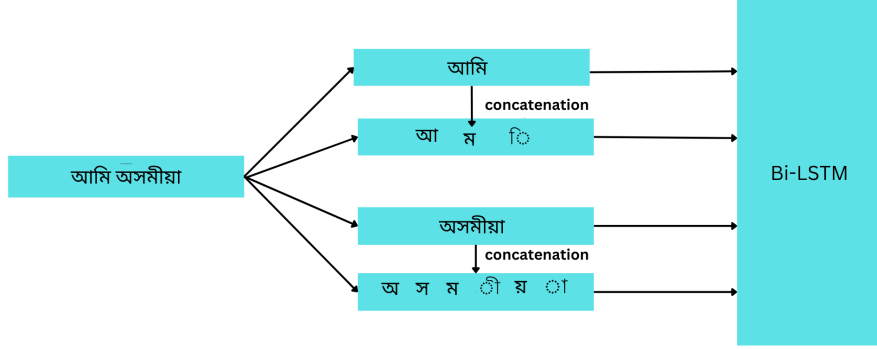


Figure 3.6: *Word-Character Tagging*

6. **Word-Bigram Tagging:** It is a combination of Word Tagging and Bigram Tagging, done simultaneously. The method of word-bigram tagging can be seen in Figure 3.7.

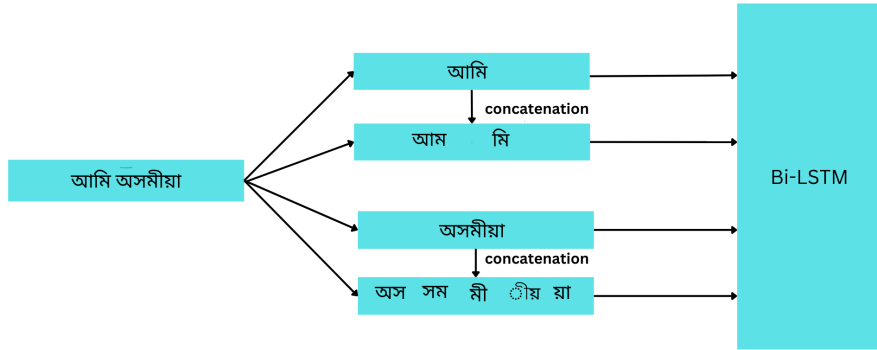


Figure 3.7: *Word-Bigram Tagging*

7. **Word-Trigram Tagging:** It is a combination of Word Tagging and Trigram Tagging, done simultaneously. The method of word-trigram tagging can be seen in Figure 3.8.

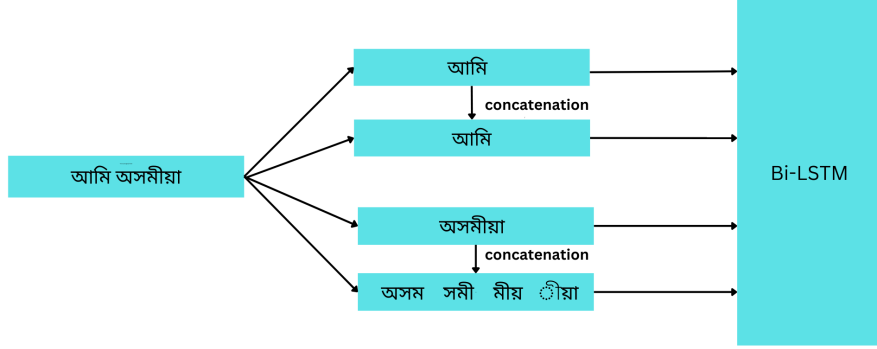


Figure 3.8: *Word-Trigram Tagging*

3.4.4 Layer 1: Alteration in Functions

The activation function utilized in [1] as well as in this project is *Softmax*. Additionally, the loss function used in this project is *Categorical Crossentropy*.

Under each method of tagging, the following experiments have been done with regard to the activation and loss functions:

1. **TensorFlow's Built-In Functions:** Here, function calls have been made to TensorFlow's built-in activation and loss function APIs.
2. **1e-2 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^2}$
3. **1e-4 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^4}$
4. **1e-6 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^6}$
5. **1e-8 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^8}$

6. **1e-10 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^{10}}$
7. **1e-12 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^{12}}$
8. **1e-14 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^{14}}$
9. **1e-16 as the ϵ value:** Here, the formula mentioned in section 3.3 has been altered by setting $\epsilon = \frac{1}{10^{16}}$

The reason behind setting this particular values of ϵ can be understood by going through Chapter 4.

3.5 Flow of Code

Below is an in-depth explanation of how the code for performing the various experiments has been written and works. It is worth noting that the flow explained below is the skeleton applied to all the different experiments. Special cases, if any, has been mentioned explicitly.

Figure 3.9 explains the overall flow of the code.

3.5.1 Importing Libraries

We start by importing necessary libraries for our code execution. TensorFlow and its Keras API are imported for building and training the neural network model. Additionally, libraries for data manipulation (Pandas, NumPy) and model evaluation (scikit-learn) are imported.

3.5.2 Importing Data

The data required for the task is imported using Pandas from the CSV file of the dataset. This data is then preprocessed to handle missing values by forward filling them.

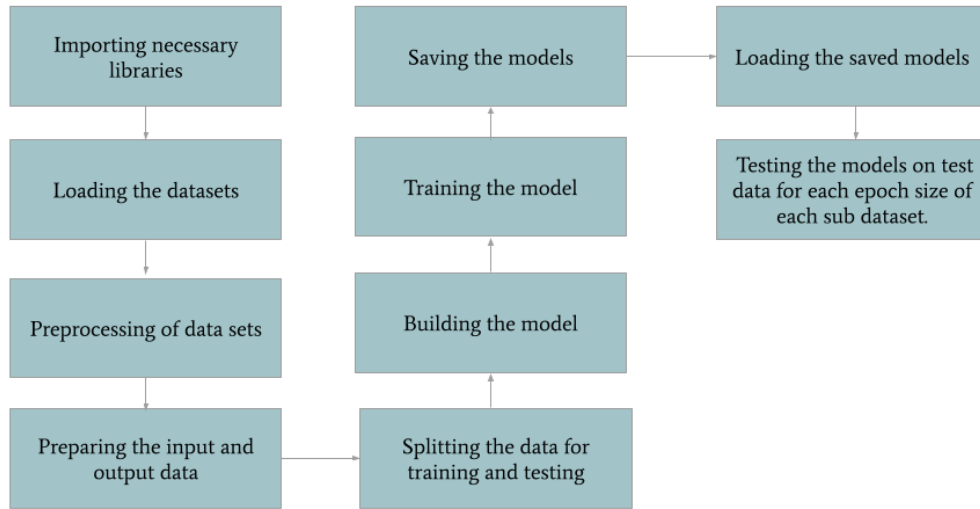


Figure 3.9: *Flow of Code*

3.5.3 Defining Classes and Functions

ReadSentences Class

This class is defined to read the sentences from the imported data. It prepares the sentences along with their corresponding words and parts of speech (POS) tags.

Custom Activation Function

A custom softmax activation function named 'custom_activation' is defined. It adjusts the input values to avoid numerical instability issues during softmax calculation. The activation custom function is coded as mentioned in Equation (3.2).

In scenarios where TensorFlow's built-in functions are used, this step is omitted.

Custom Loss Function

A custom loss function named 'custom_loss' is defined. It computes the categorical cross-entropy loss while handling edge cases to avoid numerical instability. The

custom loss function is coded as mentioned in Equation (3.4)

In scenarios where TensorFlow's built-in functions are used, this step is omitted.

Utility Functions

Utility functions for converting words to characters, bigrams or trigrams and calculating the probability distribution are defined. These functions are used for data preprocessing and analysis.

Probability Distribution Creation Function

This function creates a dictionary of words and their corresponding probabilities of tags as the key-value pairs.

For Standard approach, we omit this.

Prediction Adjustment Function

The 'adj_predict' function adjusts model predictions based on a probability distribution calculated from the training data. This adjustment is applied to improve prediction accuracy, especially for unknown words.

For Standard approach, we omit this.

EpochCheckpoint Class

This class is a custom callback for saving model checkpoints at specific epochs during training.

3.5.4 Data Preprocessing

Vocabulary and Tag Conversion

The unique words and POS tags from the data are converted into numerical indices using dictionaries ('word2id' and 'tag2id').

n-gram Vocabulary

A vocabulary of n-grams is created from the words in the dataset. This vocabulary is used to represent words as sequences of n-grams, facilitating the model to learn morphological features.

For word-only methods of tagging, this step is omitted.

3.5.5 Model Architecture

The neural network model is built using the Keras functional API. It consists of:

- Word and n-gram (if any) embedding layers to represent words and n-grams (if any) numerically.
- Bidirectional LSTM layers for capturing contextual information from both directions.
- Concatenation layer (only for a combination of word and n-gram methods) to combine word and n-gram embeddings.
- Dropout layer to prevent overfitting.
- TimeDistributed Dense layer for output prediction.

The model is compiled with the specified optimizer and loss function.

3.5.6 Training and Evaluation

The model is trained on subsets of the data with different sizes. For each subset size:

- The data is split into training and testing sets. For all the experiments, the **train-test split** is **80:20**.
- Probability distribution is calculated based on the training data. This step is ignored for Standard approach.

- Model training is performed with a custom callback to save checkpoints at specified epochs.
- After training, predictions are adjusted using the probability distribution. This step is ignored for Standard approach.
- Evaluation metrics such as accuracy are calculated for known and unknown words separately, along with overall accuracy.

The process is repeated for different subset sizes, and evaluation results are printed for each subset. The different number of **epochs** for each experiment has been set as **5, 10, 20, 30 and 50** for the **small dataset** and **5, 7, 10, 15 and 20** for the **large dataset**.

Chapter 4

Results and Observations

After following the methodology as mentioned in Chapter 3, the various results and observations under each category can be summarized as below. The metric taken into consideration is accuracy. However, for each alteration in the model, three types of accuracies have been taken into consideration:

1. **Known Accuracy:** Accuracy of prediction of known words.
2. **Unknown Accuracy:** Accuracy of prediction of unknown words.
3. **Total Accuracy:** Combination of known and unknown accuracies.

4.1 Results under Dataset 1: Small

Under the small dataset of 448 sentences, the custom models have performed in the following manner.

4.1.1 Standard Approach

By following the standard approach, the following results have been achieved under each method of tagging.

Word Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.1.

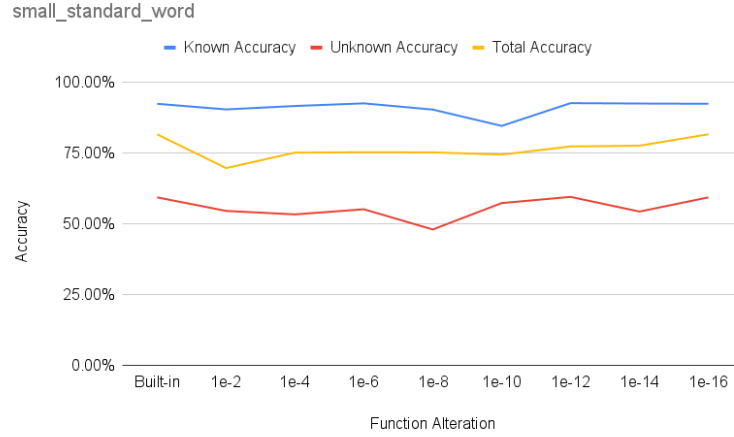


Figure 4.1: *Results of Word Tagging following Standard Approach for Small Dataset*

Character Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.2.

Bigram Tagging

For Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.3.

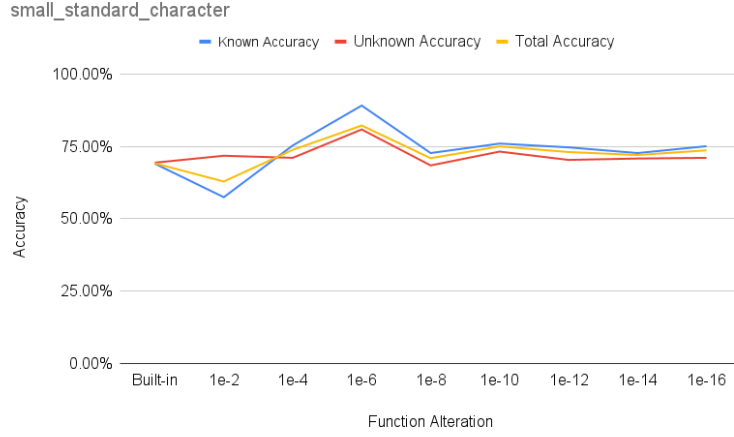


Figure 4.2: *Results of Word Tagging following Standard Approach for Small Dataset*

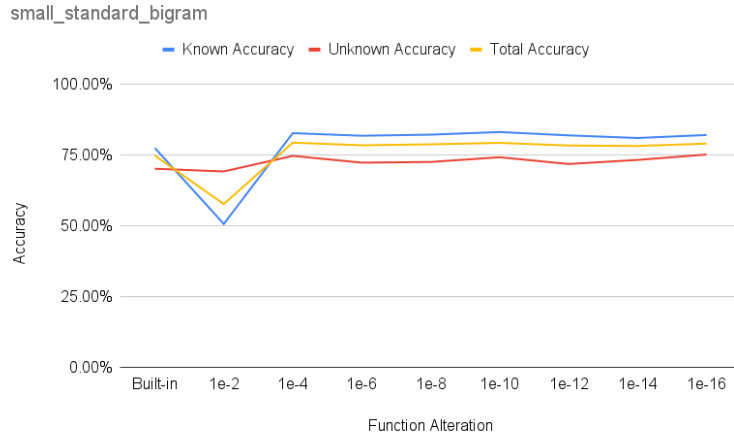


Figure 4.3: *Results of Word Tagging following Standard Approach for Small Dataset*

Trigram Tagging

For Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.4.

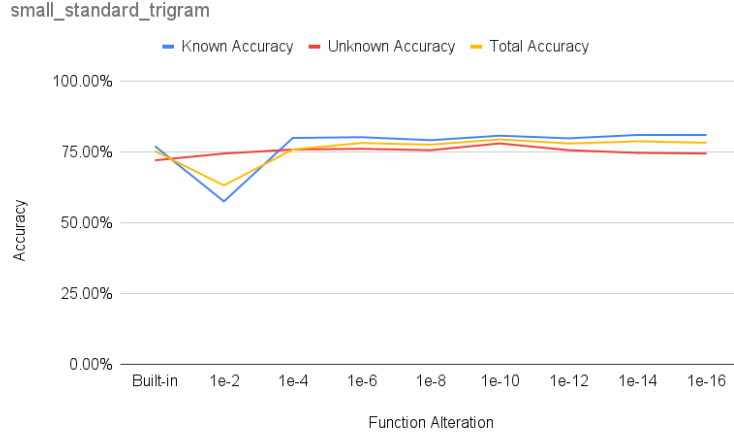


Figure 4.4: *Results of Word Tagging following Standard Approach for Small Dataset*

Word-Character Tagging

For Word-Character Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.5.

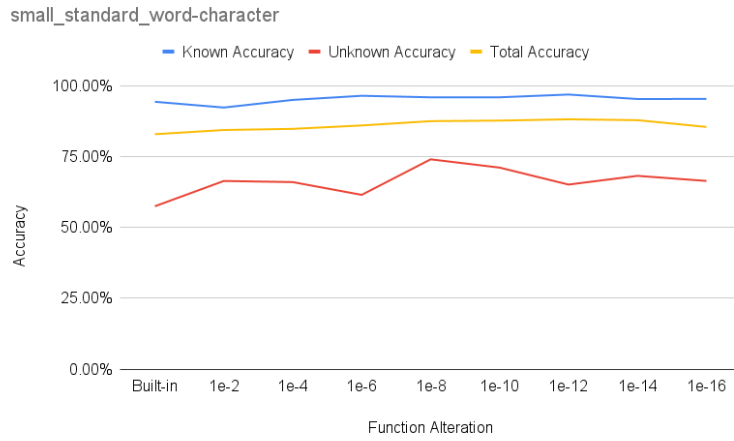


Figure 4.5: *Results of Word Tagging following Standard Approach for Small Dataset*

Word-Bigram Tagging

For Word-Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.6.

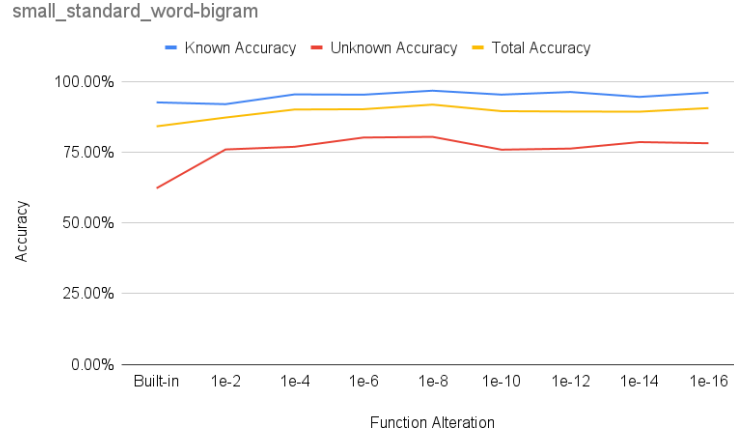


Figure 4.6: *Results of Word Tagging following Standard Approach for Small Dataset*

Word-Trigram Tagging

For Word-Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.7.

4.1.2 Probabilistic Approach

By following the Probabilistic approach, the following results have been achieved under each method of tagging.

Word Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.8.

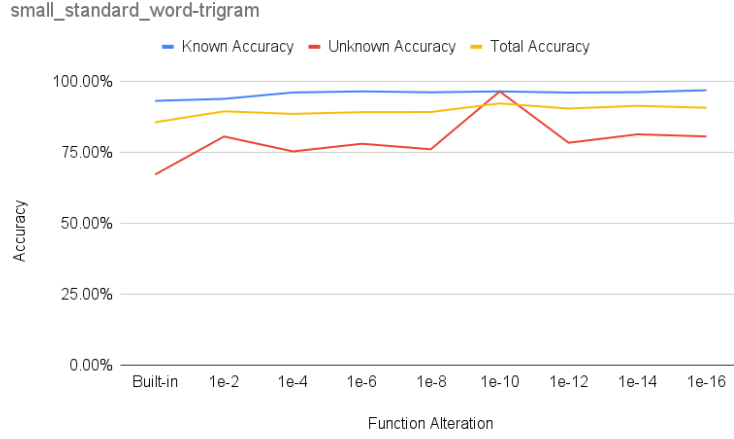


Figure 4.7: *Results of Word Tagging following Standard Approach for Small Dataset*

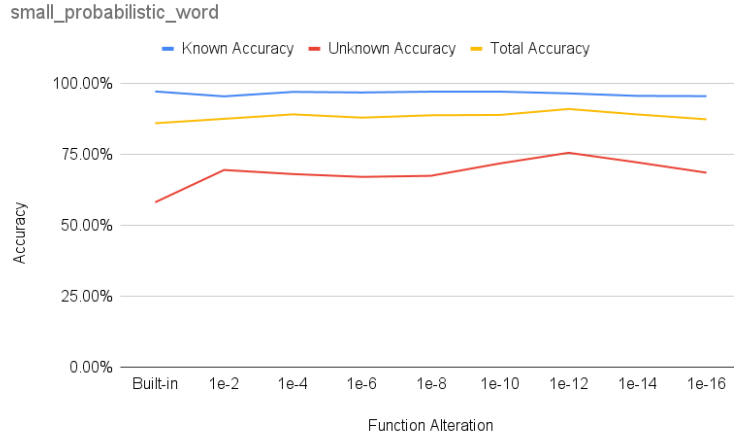


Figure 4.8: *Results of Word Tagging following Standard Approach for Small Dataset*

Character Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.9.

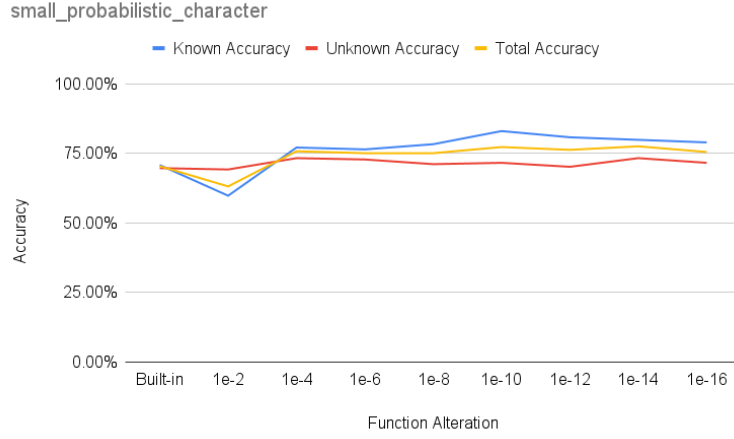


Figure 4.9: *Results of Character Tagging following Probabilistic Approach for Small Dataset*

Bigram Tagging

For Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.10.

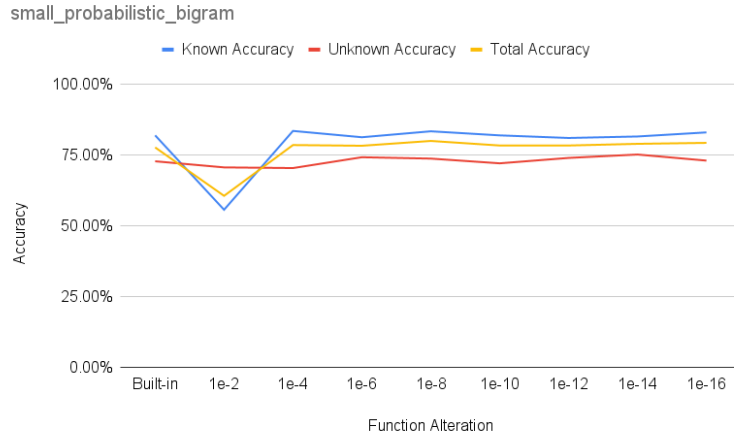


Figure 4.10: *Results of Bigram Tagging following Probabilistic Approach for Small Dataset*

Trigram Tagging

For Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.11.

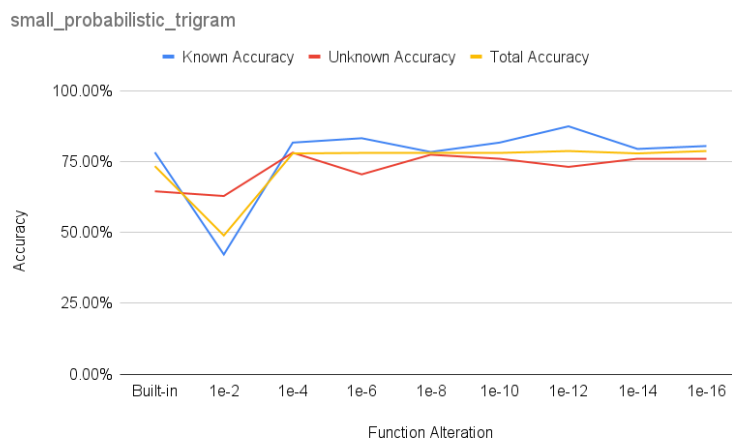


Figure 4.11: *Results of Trigram Tagging following Probabilistic Approach for Small Dataset*

Word-Character Tagging

For Word-Character Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.12.

Word-Bigram Tagging

For Word-Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.13.

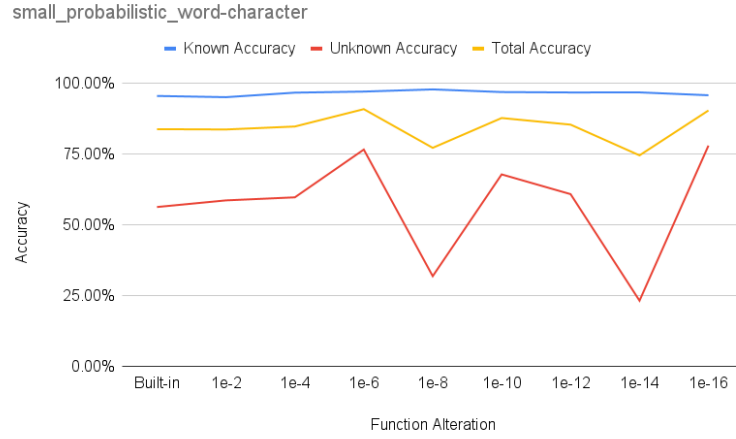


Figure 4.12: *Results of Word-Character Tagging following Probabilistic Approach for Small Dataset*

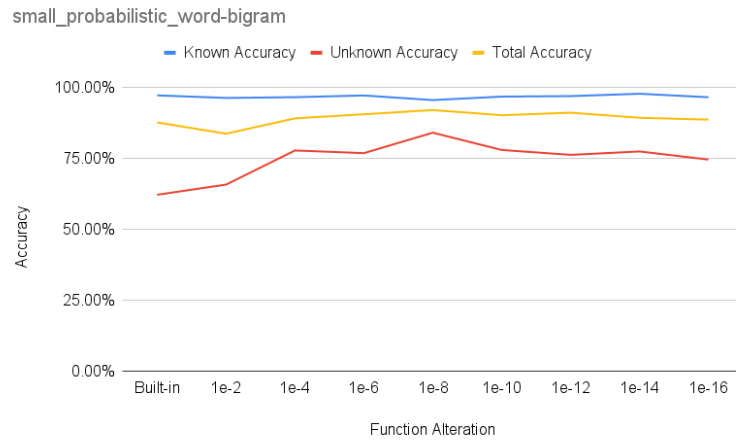


Figure 4.13: *Results of Word-Bigram Tagging following Probabilistic Approach for Small Dataset*

Word-Trigram Tagging

For Word-Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.14.

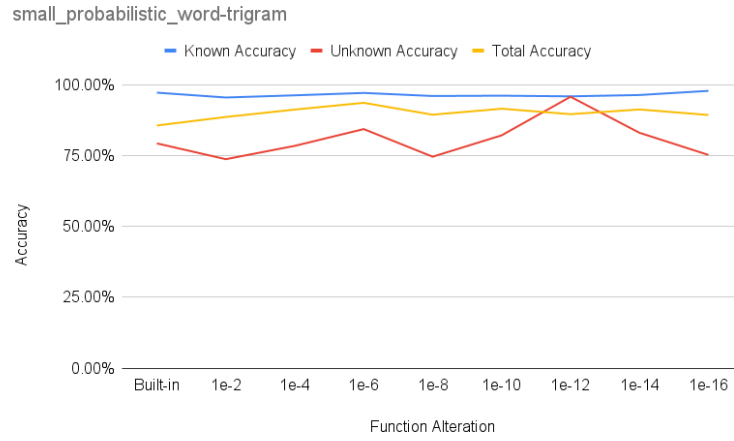


Figure 4.14: *Results of Word-Trigram Tagging following Probabilistic Approach for Small Dataset*

4.1.3 Comparison of Standard and Probabilistic Approaches

An in-depth comparative summary of the various methods of tagging and their underlying variations in the activation and loss functions can be seen in Figure 4.15.

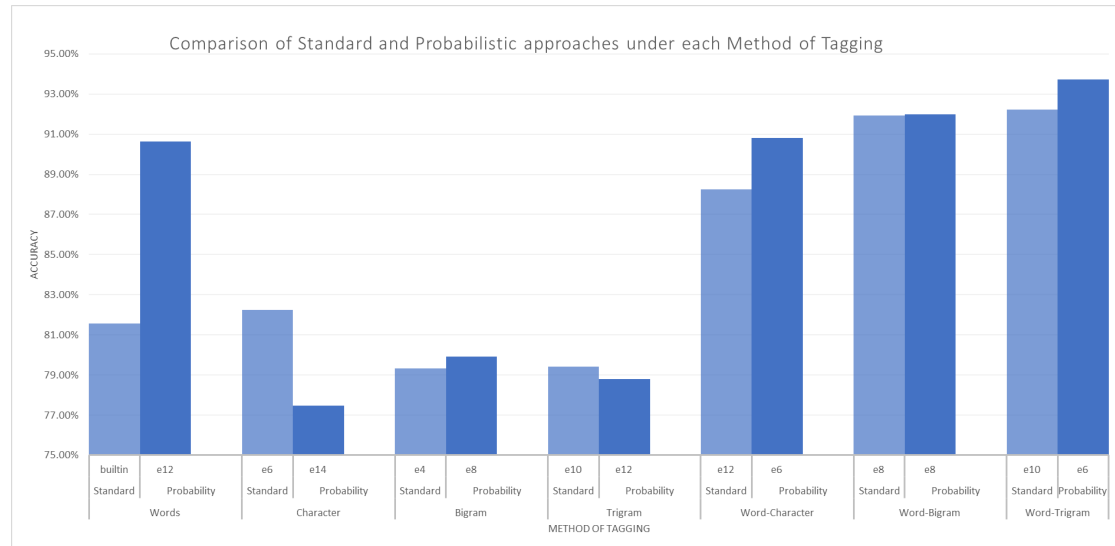


Figure 4.15: *Comparison of Standard vs Probabilistic Approach for Small dataset*

4.2 Results under Dataset 2: Large

Under the large dataset of 11,763 sentences, the custom models have performed in the following manner.

4.2.1 Standard Approach

By following the standard approach, the following results have been achieved under each method of tagging.

Word Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.16.

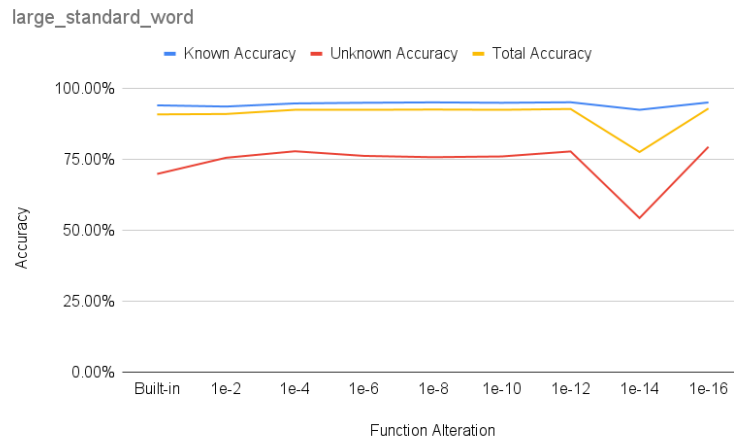


Figure 4.16: *Results of Word Tagging following Standard Approach for Large Dataset*

Character Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.17.

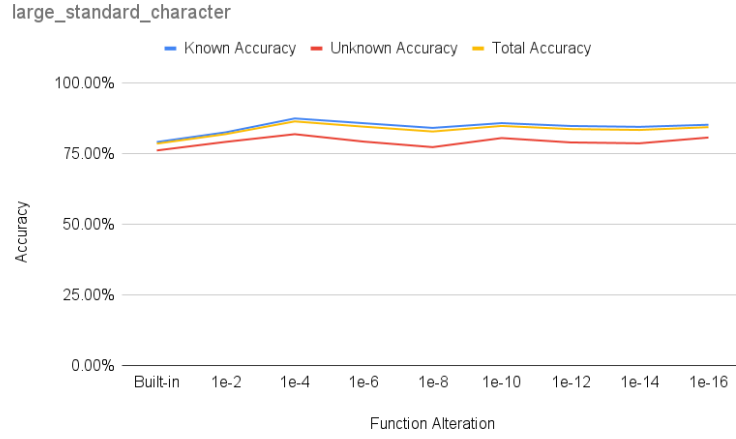


Figure 4.17: *Results of Character Tagging following Standard Approach for Large Dataset*

Bigram Tagging

For Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.18.

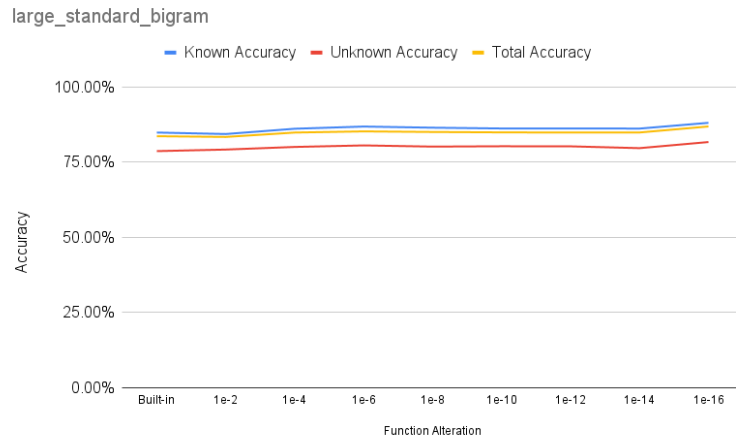


Figure 4.18: *Results of Bigram Tagging following Standard Approach for Large Dataset*

Trigram Tagging

For Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.19.

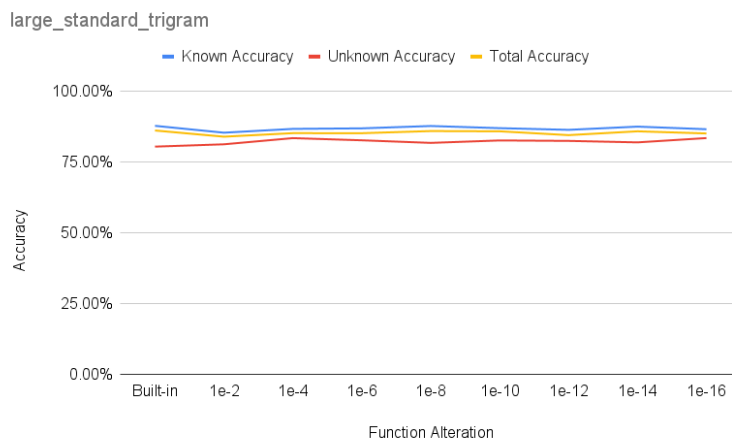


Figure 4.19: *Results of Trigram Tagging following Standard Approach for Large Dataset*

Word-Character Tagging

For Word-Character Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.20.

Word-Bigram Tagging

For Word-Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.21.

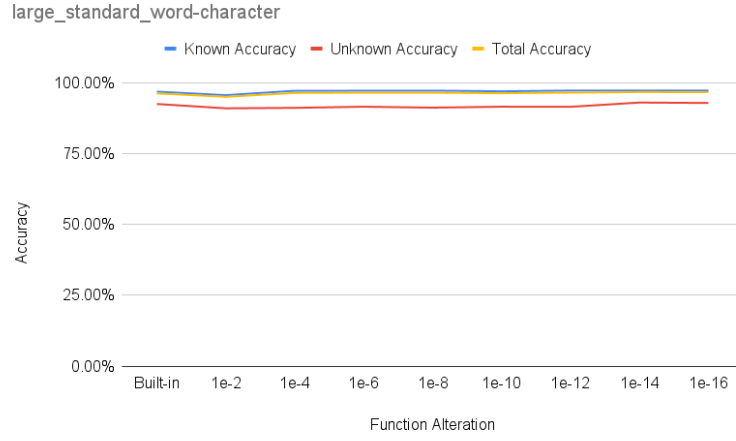


Figure 4.20: *Results of Word-Character Tagging following Standard Approach for Large Dataset*

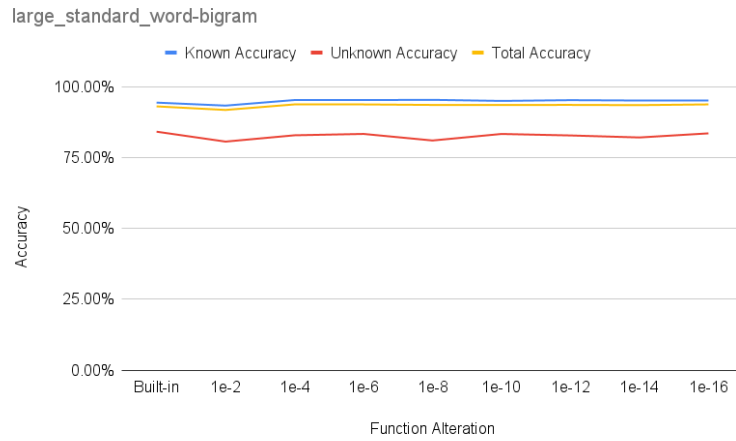


Figure 4.21: *Results of Word-Bigram Tagging following Standard Approach for Large Dataset*

Word-Trigram Tagging

For Word-Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.22.

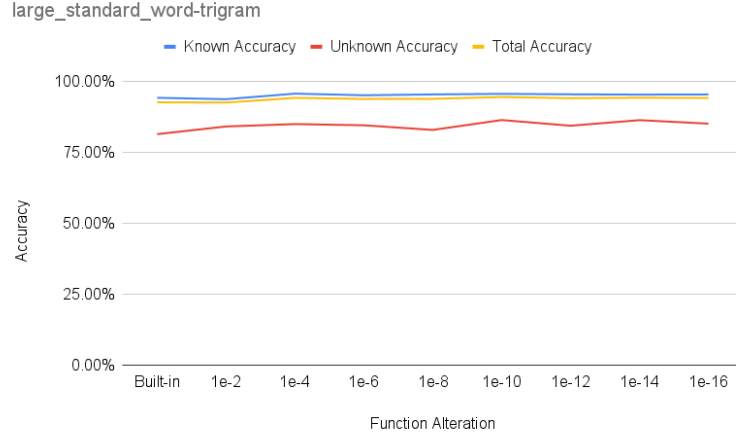


Figure 4.22: *Results of Word-Trigram Tagging following Standard Approach for Large Dataset*

4.2.2 Probabilistic Approach

By following the Probabilistic approach, the following results have been achieved under each method of tagging.

Word Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.23.

Character Tagging

For Word Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.24.

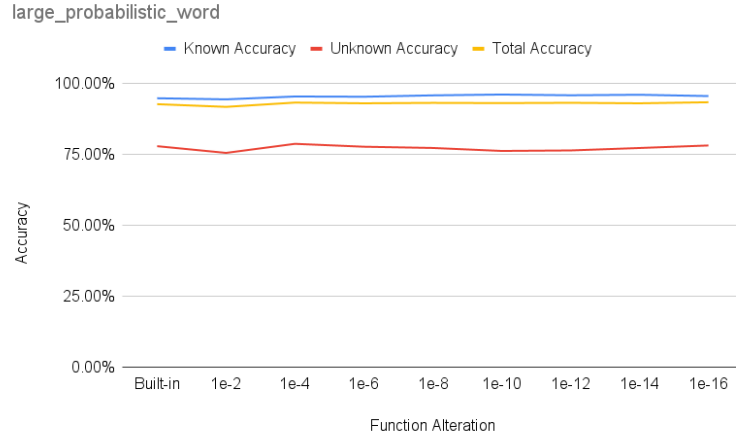


Figure 4.23: *Results of Word Tagging following Probabilistic Approach for Large Dataset*

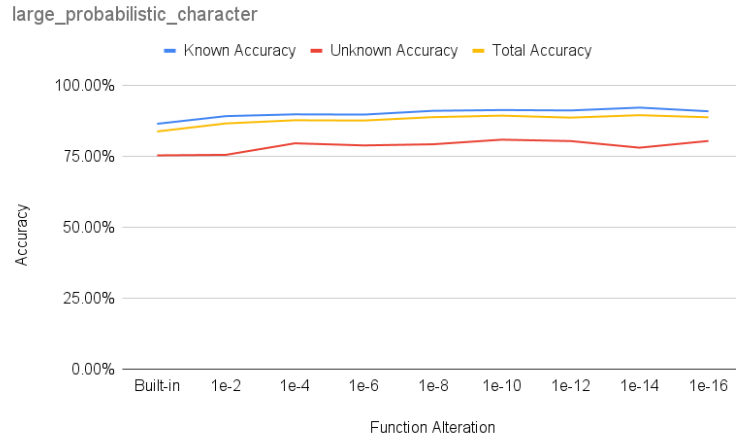


Figure 4.24: *Results of Character Tagging following Probabilistic Approach for Large Dataset*

Bigram Tagging

For Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.25.

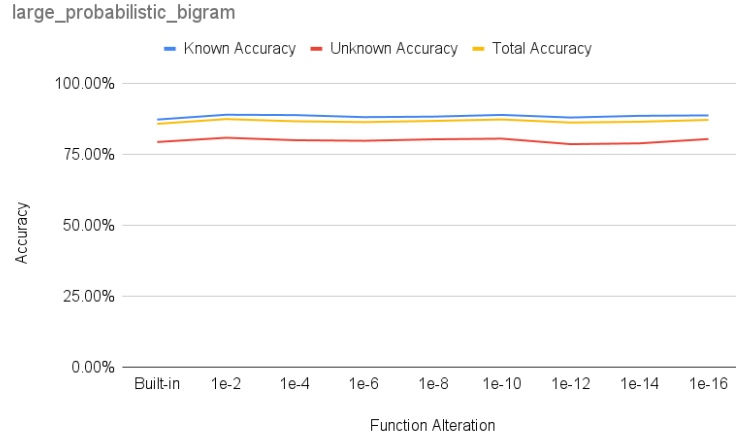


Figure 4.25: *Results of Bigram Tagging following Probabilistic Approach for Large Dataset*

Trigram Tagging

For Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.26.

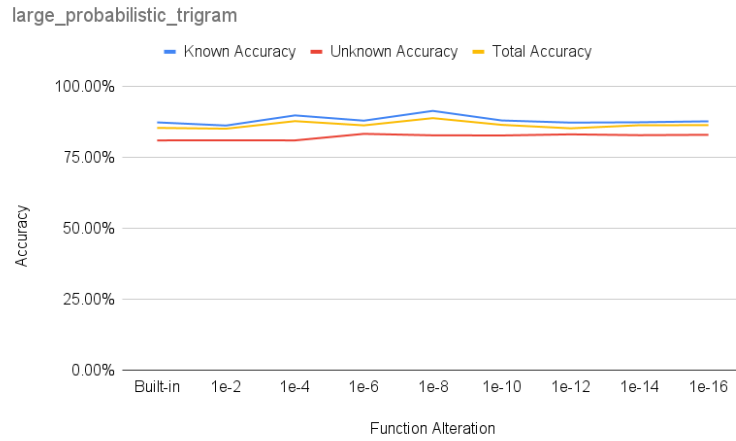


Figure 4.26: *Results of Trigram Tagging following Probabilistic Approach for Large Dataset*

Word-Character Tagging

For Word-Character Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.27.

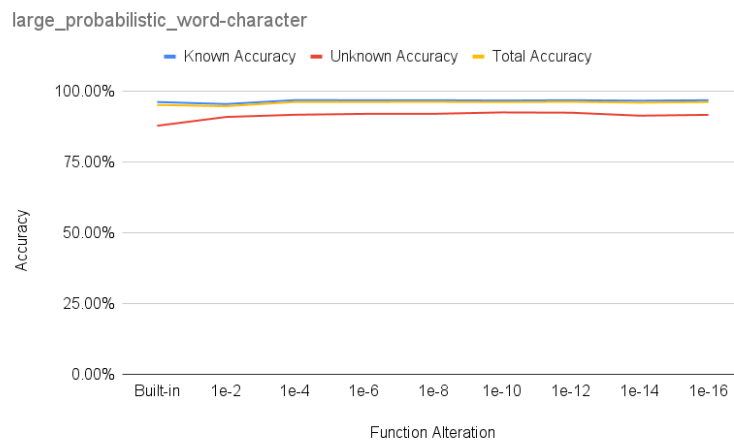


Figure 4.27: *Results of Word-Character Tagging following Probabilistic Approach for Large Dataset*

Word-Bigram Tagging

For Word-Bigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.28.

Word-Trigram Tagging

For Word-Trigram Tagging, the results obtained by each of the ϵ values, ranging from $\frac{1}{10^2}$ to $\frac{1}{10^{16}}$, as well as their comparison with the built-in TensorFlow functions can be seen in Figure 4.29.

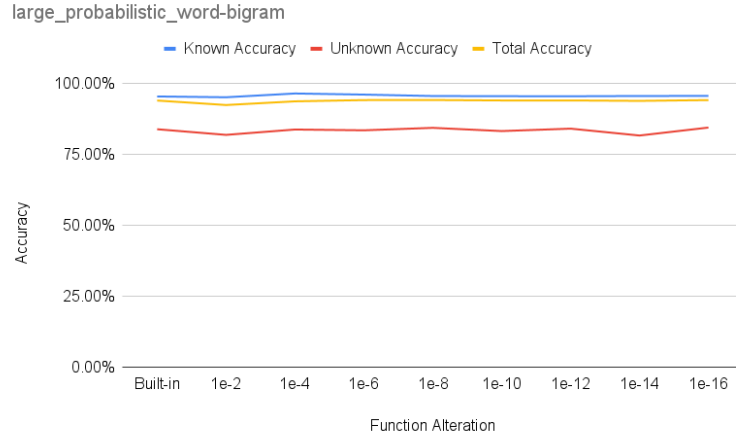


Figure 4.28: *Results of Word-Bigram Tagging following Probabilistic Approach for Large Dataset*

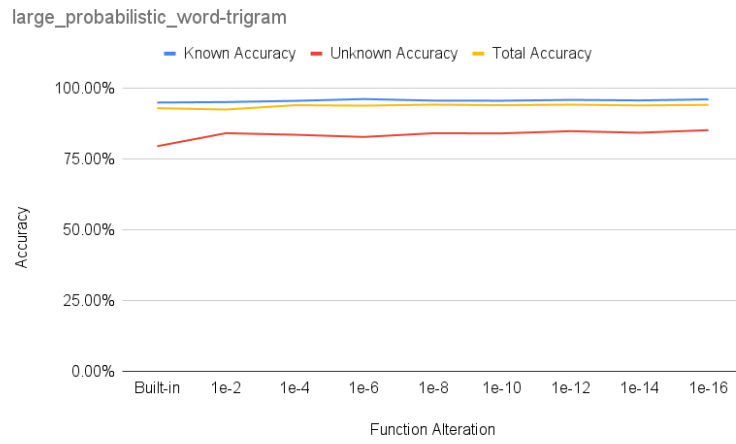


Figure 4.29: *Results of Word-Trigram Tagging following Probabilistic Approach for Large Dataset*

4.2.3 Comparison of Standard and Probabilistic Approaches

An in-depth comparative summary of the various methods of tagging and their underlying variations in the activation and loss functions can be seen in Figure 4.30.

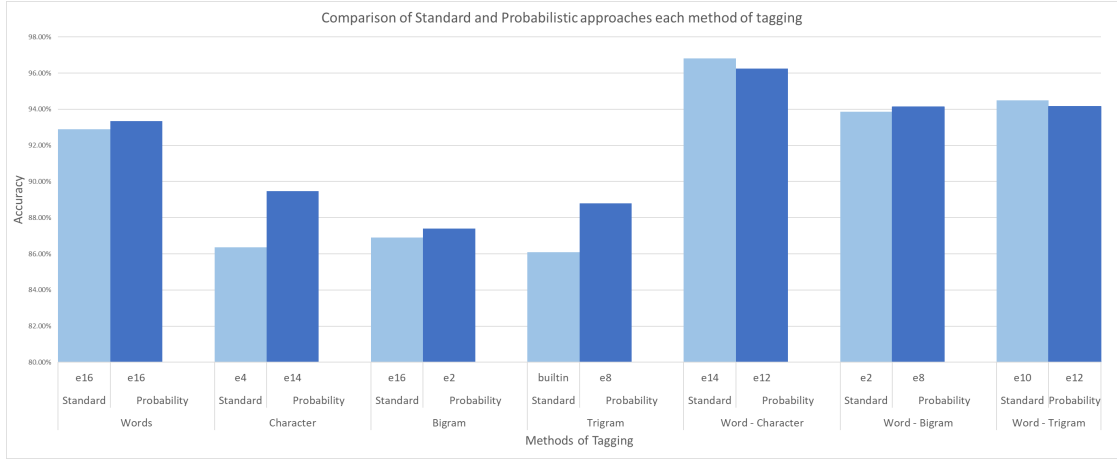


Figure 4.30: *Comparison of Standard vs Probabilistic Approach for Large dataset*

4.3 Observations

After a thorough analysis of the various results obtained under each of the categories, sub-categories and approaches, the following observations have been discovered.

4.3.1 Dataset 1 (Small)

1. The Probabilistic approach performs significantly better than the Standard approach in most scenarios.
2. The Word-Trigram method of tagging is observed to be performing the best.
3. The models having ϵ value between $\frac{1}{10^8}$ and $\frac{1}{10^{12}}$ give us the most optimum results.

4.3.2 Dataset 2 (Large)

1. The Probabilistic approach performs significantly better than the Standard approach in most scenarios.

2. The Word-Character method of tagging is observed to be performing the best.
3. The models having ϵ value between $\frac{1}{10^8}$ and $\frac{1}{10^{14}}$ give us the most optimum results.

4.3.3 General Observations

1. The study finds that the using a Probabilistic approach for training and testing POS tagging models give the most optimum results.
2. The most optimum method of tagging is the concatenation of words and n-grams.
3. The smaller the value of ϵ , the better is the accuracy. However, the value should not be too small, it should be in the range: $\frac{1}{10^8} < \epsilon < \frac{1}{10^{14}}$.
4. The implemented custom activation, PSM-Taylor Softmax activation function and the proposed loss functions outperform the built-in TensorFlow functions by a huge margin in almost all the scenarios.
5. A probabilistic approach for deep learning is particularly advantageous for POS tagging because it allows the model to capture the inherent uncertainty and variability in natural language. By leveraging probability distributions, probabilistic models can account for the multiple possible tags that a word might have based on its context, leading to more accurate and robust predictions.
6. The word-n-gram method is often preferred for POS tagging due to its ability to capture contextual information effectively while maintaining simplicity and efficiency. By considering sequences of words of length n (n-grams), this method leverages local context to predict the POS tag of each word. This approach allows the model to capture dependencies between adjacent words, which are crucial for accurate tagging. Additionally, word-n-gram models are computationally efficient and require relatively small amounts

of memory compared to more complex models, making them suitable for large-scale POS tagging tasks.

7. Subtracting a small constant ϵ from the input tensor x before applying exponential operation is a technique commonly used to improve numerical stability, especially when dealing with deep learning models and softmax calculations. In deep learning models, the values in the input tensor x can be large, especially during training. When these large values are exponentiated, they can lead to overflow issues, causing numerical instability or NaN values. Subtracting a small constant ϵ helps prevent such overflow by shifting the values closer to zero, which makes the models more efficient.
8. Normalizing the loss by the number of inputs ensures that the magnitude of the loss is consistent regardless of the batch size. Without normalization, the loss would scale linearly with the batch size, which could make it challenging to compare models trained on datasets with different batch sizes. That is why the custom loss function performs better.

Chapter 5

Application of Results

It has been found that the results of this project have performed very well in datasets of varying sizes. To make people around the world able to access the benefits of this project, a web-based application has been created using the Flask framework of Python.

The directory-file structure of the application can be viewed in Figure 5.1.

```
assamese-pos-tagger
├── static
│   └── css
│       └── index.css
├── templates
│   ├── icons8-github.svg
│   └── index.html
├── .gitignore
├── app.py
├── model_config.json
├── large_dataset.csv
├── requirements.txt
└── large_model.h5
```

Figure 5.1: *File structure of web application*

5.1 About

Following are the primary frameworks/services that have been utilized for the development and deployment of this application.

5.1.1 Flask

Flask is a micro web framework written in Python. It is classified as a microframework because it does not require particular tools or libraries. It has no database abstraction layer, form validation, or any other components where pre-existing third-party libraries provide common functions. However, Flask supports extensions that can add application features as if they were implemented in Flask itself. Extensions exist for object-relational mappers, form validation, upload handling, various open authentication technologies and several common framework related tools. Flask is used by enterprises all over the world, including Pinterest, LinkedIn etc.

5.1.2 Render

Render is a cloud hosting provider that can run every shape and size of application: static websites, dynamic web apps, scheduled cron jobs etc. It natively supports many popular programming languages, like Node.js, Python, and Ruby. Render provides a Free tier for hobbyists, students, and experimenters. Render takes care of deploying and running user apps so that they can focus on creating them. It gives every web app its own `onrender.com` URL. Render also handles a lot of the tricky networking concerns that come with app hosting, like managing TLS certificates.

5.2 Development of Application

5.2.1 Libraries and Setup

Import Statements:

- **tensorflow as tf:** Import TensorFlow for neural network functionalities.
Flask, render_template, request: Import Flask and its functionalities to create a web server and handle requests.
- **nltk:** Import the Natural Language Toolkit for tokenizing sentences.
- **numpy as np:** Import NumPy for numerical operations.
- **pandas as pd:** Import Pandas for data manipulation.

Download NLTK Data:

`nltk.download('punkt')` downloads the Punkt tokenizer model, necessary for tokenizing sentences.

5.2.2 Flask App Initialization

Initialize Flask App:

- `app = Flask(__name__)` initializes the Flask application.

5.2.3 Data Loading and Processing

Load Dataset:

- `data = pd.read_csv('large_dataset.csv', encoding='utf-8')`: Reads the CSV file containing words and their corresponding POS tags.
- `data = data.fillna(method="ffill")`: Fills any missing values using forward fill method.

Extract Unique Tags and Words:

- **tags = list(set(data["Pos"].values)):** Extracts unique POS tags from the dataset.
- **words = list(set(data["Word"].values)):** Extracts unique words from the dataset.

Create Mappings:

- **word2id = w: i for i, w in enumerate(words):** Creates a dictionary mapping each word to a unique integer ID.
- **tag2id = t: i for i, t in enumerate(tags):** Creates a dictionary mapping each POS tag to a unique integer ID.

5.2.4 Model Configuration and Loading

Model Configuration:

The **config** dictionary defines the architecture of the neural network model, which includes:

- Input layer
- Embedding layer
- Dropout layer
- Bidirectional LSTM layer
- TimeDistributed Dense layer with softmax activation

Load Pre-trained Model:

- **model = tf.keras.models.Model.from_config(config):** Initializes the model using the defined configuration.
- **model.load_weights("your_model_file.h5"):** Loads the pre-trained weights into the model.

5.2.5 Flask Route for Handling Requests

Define Home Route:

- `@app.route('/', methods=['GET', 'POST'])` defines the home route which handles both GET and POST requests.

Handle POST Requests:

- `if request.method == 'POST':` Checks if the request method is POST.
- `user_input = request.form['sentence']:` Retrieves the sentence input from the form.
- `tokenized_sentence = nltk.word_tokenize(user_input):` Tokenizes the input sentence into words.
- `X_Samp = [[word2id.get(word, len(words) - 1) for word in tokenized_sentence]]:` Converts each word to its corresponding ID, using the last ID for unknown words.
- `X_Samp = np.array(X_Samp):` Converts the list to a NumPy array.

Predict POS Tags:

- `with tf.device('/cpu:0'):` Ensures the prediction runs on the CPU.
- `p = model.predict(X_Samp):` Uses the model to predict the POS tags.
- `p = np.argmax(p, axis=-1):` Converts the softmax outputs to their corresponding tag IDs.

Create Tagged Sentence:

- `tagged_sentence = [(word, tags[pred]) for word, pred in zip(tokenized_sentence, p[0])]:` Maps each word to its predicted POS tag.

Render Template:

- **return render_template('index.html', sentence=sentence, tagged_sentence = tagged_sentence):** Renders the index.html template, passing the original sentence and the tagged sentence as variables.

5.2.6 HTML Template

index.html: The HTML template file would contain the form for inputting sentences and displaying the tagged output. This is not included in the code but is essential for the web interface.

5.3 Deployment of Application

The deployment of the web-based application has been done on the cloud-hosting platform known as Render. Here are the steps followed for the deployment:

5.3.1 Preparing the Flask App for Deployment

1. Finalizing the project structure to suit the deployment.
2. Creating the '**requirements.txt**' file and saving the versions of all the dependencies, libraries and packages there.
3. Modifying the main '**app.py**' file for deployment.

5.3.2 Pushing Code to GitHub

1. Creating a '**.gitignore**' file to avoid committing unnecessary and/or sensitive files.
2. Initializing a Git Repository.
3. Committing the code.
4. Creating a GitHub repository.
5. Pushing the local repository to GitHub.

5.3.3 Deploying on Render

1. Creating a Render account.
2. Creating a new '**Web Service**' in the platform.
3. Connecting GitHub account to Render.
4. Configure the service by specifying its name, region, branch, root directory etc.
5. Specifying the build and start commands.
6. Clicking on '**Create Web Service**' to start the deployment process.

5.3.4 Accessing the App

1. Checking build and deployment status.
2. Accessing the app via the URL generated after successful completion of deployment.

5.4 Working of Application

The working of the application can be viewed via Figure 5.2 and Figure 5.3. Furthermore, the GitHub link of the project and the domain of the deployment is as follows:

- **GitHub:** <https://github.com/aditya1ojha/postagger-assamese>
- **Render Deployment:** <https://assamese-partofspeech-tagger.onrender.com/>

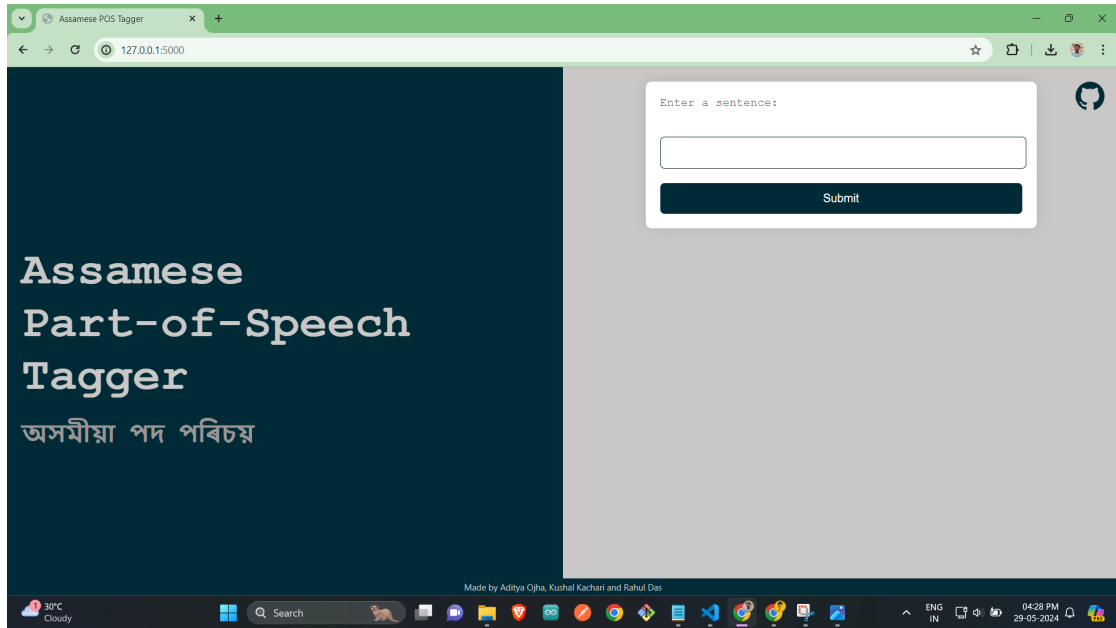


Figure 5.2: *Landing Page*

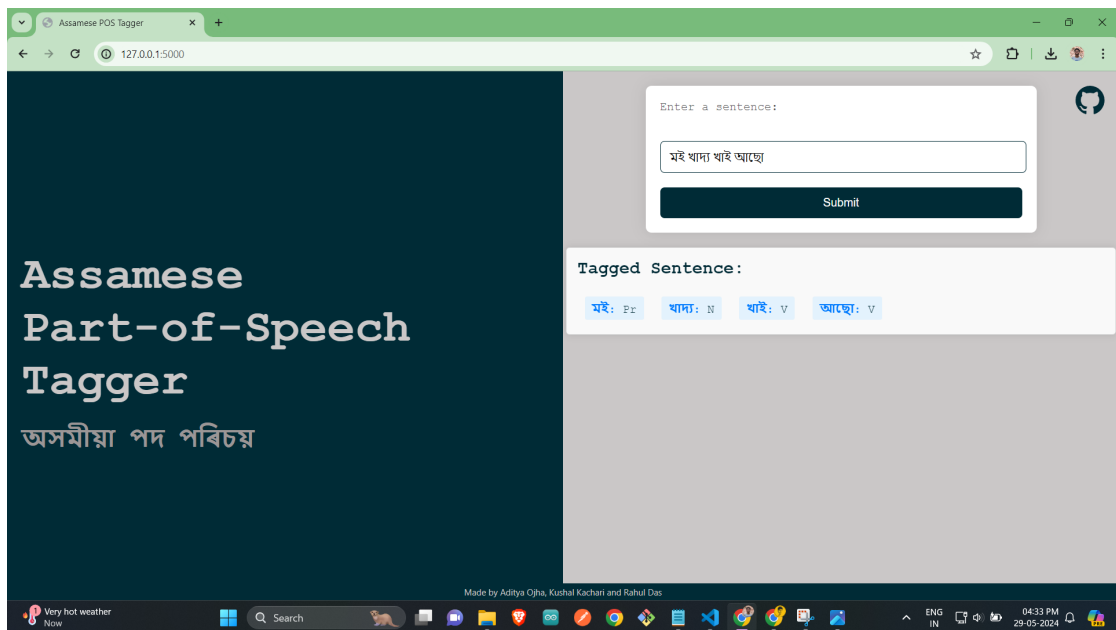


Figure 5.3: *Tagged Sentence*

Chapter 6

Future Prospects

The project in its current state is very robust and works well in real-life scenarios. However, there is always some room for improvement. Hence, following is the list of tasks that are planned to be done with respect to this project:

1. Writing and publishing a research paper by extending the ideas of [1] and combining the findings that have been observed in this project.
2. Making the web application capable of dealing with files as well.
3. Making the web application more dynamic by storing the data given as input by the users and re-training and re-deploying the models at regular intervals of time.
4. Exploring in-demand architecture for NLP like transformers and LLMs and making the models more reliable in real-life scenarios.

Chapter 7

Conclusion

The project "*A Comprehensive Study and Implementation of Part-of-speech Tagging in the Assamese Language*" successfully addressed the gap in natural language processing resources for the language of Assamese by developing a robust part-of-speech (POS) tagging system. Through an extensive literature review, dataset preprocessing, and experimentation with various tagging approaches, the project identified the value of $\epsilon = \frac{1}{10^{12}}$ in the PSM-Taylor activation function, under Word-n-gram method of tagging, under the Probabilistic approach to be the most efficient. The inclusion of language-specific features, such as handling inflections and compound words, significantly enhanced the tagger's performance. Most importantly, the project served as an adequate implementation of the studies previously conducted; and the findings outperformed the most widely used existing activation and loss functions available to use. Additionally, the deployment of the easy-to-use web application on the Internet makes anyone in the world capable to take advantage of the results concluded in this project. This project not only created a reliable POS tagging tool but also highlighted the importance of tailored linguistic resources for underrepresented languages. At last but not the least, the project served as a great learning opportunity and experience to obtain an unprecedented in-depth understanding in the field of Natural Language Processing and Deep Learning approaches for Part-of-Speech Tagging.

References

- [1] Diganta Baishya and Rupam Baruah, *Part-of-speech Tagging for Low-resource Languages: Activation Function for Deep Learning Network to Work with Minimal Training Data*, ACM Trans. Asian Low-Resour. Lang. Inf. Process., Vol. 23, No. 5, Article 70 (2024).
- [2] Angel R. Martinez, *Part-of-speech tagging*, Wiley Interdiscip. Rev.: Computat. Stat. 4, 1 (2012), 107–113.
- [3] Lluís Marquez, Lluís Padro, and Horacio Rodríguez, *A machine learning approach to POS tagging*, Mach. Learn. 39 (2000), 59–91.
- [4] Bao-Liang Lu, Qing Ma, Michinori Ichikawa, and Hitoshi Isahara, *Efficient part-of-speech tagging with a min-max modular neural-network model*, Appl. Intell. 19 (2003), 65–81.
- [5] Cícero Nogueira dos Santos and Bianca Zadrozny, *Learning character-level representations for part-of-Speech Tagging*, International Conference on Machine Learning, 2014.
- [6] A. Chiche and B. Yitagesu, *Part of speech tagging: A systematic review of deep learning and machine learning approaches*, J. Big Data 9, 10 (2022). DOI: <https://doi.org/10.1186/s40537-022-00561-y>
- [7] Richard Socher, Yoshua Bengio, and Christopher D. Manning, *Deep Learning for NLP (without Magic)*, Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts, page 5, Jeju Island, Korea, Association for Computational Linguistics, 2012.

- [8] Tanya Dayanand, *POS tagging using RNN*, Towards Data Science, 2020. Retrieved from <https://towardsdatascience.com/pos-tagging-using-rnn-7f08a522f849>
- [9] S. K. Ahammad Fahad and Abdulsamad Ebrahim Yahya, *Inflectional review of deep learning on natural language processing*, International Conference on Smart Computing and Electronic Enterprise (ICSCEE'18), IEEE, 2018.
- [10] Tobias Horsmann and Torsten Zesch, *Do LSTMs really work so well for PoS tagging?—A replication study*, Conference on Empirical Methods in Natural Language Processing, 2017.
- [11] Barbara Plank, Anders Søgaard, and Yoav Goldberg, *Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss*, arXiv preprint arXiv:1604.05529 (2016).
- [12] Manoj Kumar Balwant, *Bidirectional LSTM based on POS tags and CNN architecture for fake news detection*, 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT'19), IEEE, 2019.
- [13] Xiaorui Xue and Jiansong Zhang, *Part-of-speech tagging of building codes empowered by deep learning and transformational rules*, Adv. Eng. Inform. 47 (2021), 101235.
- [14] Xiaoqing Zheng, Hanyang Chen, and Tianyu Xu, *Deep learning for Chinese word segmentation and POS tagging*, Conference on Empirical Methods in Natural Language Processing, 2013.
- [15] D. Baishya and R. Baruah, *Improving hidden markov model for very low resource languages: An analysis for Assamese parts of speech tagging*, 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence), IEEE, 2021, 142–146.
- [16] Diganta Baishya and Rupam Baruah, *Highly efficient parts of speech tagging in low resource languages with improved hidden Markov model and deep learning*, Int. J. Advanc. Comput. Sci. Applic. 12, 10 (2021).

- [17] Dhumal Deshmukh and A. Kiwelekar, *Deep learning techniques for part of speech tagging by natural language processing*, 2nd International Conference on Innovative Mechanisms for Industry Applications (ICIMIA'20), 2020, 76–81. DOI: <https://doi.org/10.1109/ICIMIA48430.2020.9074941>
- [18] W. Nelson Francis and Henry Kucera, *Brown Corpus Manual: Manual of Information to Accompany a Standard Corpus of Present-day Edited American English for Use with Digital Computers*, Retrieved from <http://icame.uib.no/brown/bcm.html>, 2001.
- [19] John Lafferty, McCallum Andrew, and Pereira Fernando, *Conditional random fields: Probabilistic models for segmenting and labeling sequence data*, ICML 1, 2 (2001), 3.
- [20] Trevor A. Cohn, *Scaling conditional random fields for natural language processing*, PhD Thesis, University of Melbourne, Department of Computer Science and Software Engineering, Faculty of Engineering, 2007.
- [21] Charles A. Sutton, *Efficient training methods for conditional random fields*, Doctoral Dissertations Available from Proquest. AAI3315485. <https://scholarworks.umass.edu/dissertations/AAI3315485>, 2008.
- [22] Zhiheng Huang, Wei Xu, and Kai Yu, *Bidirectional LSTM-CRF models for sequence tagging*, arXiv preprint arXiv:1508.01991 (2015).
- [23] Y. Chen, K. Wang, W. Yang, Y. Qing, R. Huang and P. Chen, *A multi-channel deep neural network for relation extraction*, IEEE Access 8 (2020), 13195–13203. DOI: <https://doi.org/10.1109/ACCESS.2020.2966303>
- [24] W. Khan, A. Daud, K. Khan, J. A. Nasir, M. Basher, N. Aljohani, and F. S. Alotaibi, *Part of speech tagging in Urdu: Comparison of machine and deep learning approaches*, IEEE Access 7 (2019), 38918–38936. DOI: <https://doi.org/10.1109/ACCESS.2019.2897327>
- [25] J. Kim, Y. Ko, and J. Seo, *A bootstrapping approach with CRF and deep learning models for improving the biomedical named entity recog-*

- nitition in multi-domains*, IEEE Access 7 (2019), 70308–70318. DOI: <https://doi.org/10.1109/ACCESS.2019.2914168>
- [26] V. Sornlertlamvanich and S. Yuenyong, *Thai named entity recognition using BiLSTM-CNN-CRF enhanced by TCC*, IEEE Access 10 (2022), 53043–53052. DOI: <https://doi.org/10.1109/ACCESS.2022.3175201>
- [27] Wasan AlKhawter and Nora Al-Twairish, *Part-of-speech tagging for Arabic tweets using CRF and Bi-LSTM*, Comput. Speech Lang. 65 (2021), 101138.
- [28] Imad Zeroual, Abdelhak Lakhouaja, and Rachid Belahbib, *Towards a standard part of speech tagset for the Arabic language*, J. King Saud Univ.-Comput. Inf. Sci. 29, 2 (2017), 171–178.
- [29] T. S. Nguyen, L. M. Nguyen, S. Tojo, K. Satoh, and A. Shimazu, *Recurrent neural network-based models for recognizing requisite and effectuation parts in legal texts*, Artif. Intell. Law 26 (2018), 169–199.
- [30] N. K. Senthil Kumar and N. Malarvizhi, *Bi-directional LSTM–CNN combined method for sentiment analysis in part of speech tagging (PoS)*, Int. J. Speech Technol. 23 (2020), 373–380.
- [31] Sara Besharati, Hadi Veisi, Ali Darzi, and Seyed Habib Hosseini Saravani, *A hybrid statistical and deep learning based technique for Persian part of speech tagging*, Iran J. Comput. Sci. 4 (2021), 35–43.
- [32] S. Nagesh Bhattu, Satya Krishna Nunna, Durvasula V. L. N. Somayajulu, and Binay Pradhan, *Improving code-mixed POS tagging using code-mixed embeddings*, ACM Trans. Asian Low-resour. Lang. Inf. Process. 19, 4 (2020), 1–31.
- [33] Sunita Warjri, Partha Pakray, Saralin A. Lyngdoh, and Arnab Kumar Maji, *Part-of-Speech (POS) tagging using deep learning-based approaches on the designed Khasi POS corpus*, Trans. Asian Low-resour. Lang. Inf. Process. 21, 3 (2021), 1–24.
- [34] Kunal Banerjee, Vishak C., Rishi Raj Gupta, Kartik Vyas, Anushree H., and Biswajit Mishra, *Exploring alternatives to softmax function*, ArXiv abs/2011.11538 (2020).

- [35] W. Shi, Y. Gong, X. Tao, D. Cheng, and N. Zheng, *Fine-grained image classification using modified DCNNs trained by cascaded SoftMax and generalized large-margin losses*, IEEE Trans. Neural Netw. Learn. Syst. 30, 3 (Mar. 2019), 683–694. DOI: <https://doi.org/10.1109/TNNLS.2018.2852721>
- [36] Y. Luo, Y. Wong, M. Kankanhalli and Q. Zhao, *G-SoftMax: Improving intraclass compactness and interclass separability of features*, IEEE Trans. Neural Netw. Learn. Syst. 31, 2 (Feb. 2020), 685–699. DOI: <https://doi.org/10.1109/TNNLS.2019.2909737>
- [37] F. Wang, J. Cheng, W. Liu, and H. Liu, *Additive margin SoftMax for face verification*, IEEE Sig. Process. Lett. 25, 7 (July 2018), 926–930. DOI: <https://doi.org/10.1109/LSP.2018.2822810>
- [38] Weiyang Liu, Yandong Wen, Zhiding Yu, and Meng Yang, *Large-Margin SoftMax Loss for Convolutional Neural Networks*, ArXiv abs/1612.02295 (2016).
- [39] Pascal Vincent, Alexandre De Brébisson, and Xavier Bouthillier, *Efficient exact gradient update for training deep networks with very large sparse targets*, Adv. Neural Inf. Process. Syst. 28, (2015).
- [40] Xuezhi Liang, Xiaobo Wang, Zhen Lei, Shengcai Liao, and Stan Z. Li, *Soft-margin SoftMax for deep classification*, International Conference on Neural Information Processing. Springer International Publishing, 2017.
- [41] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, *Natural language processing (almost) from scratch*, J. Mach. Learn. Res. 12 (2011), 2493–2537.
- [42] Katharina Kann, Johannes Bjerva, Isabelle Augenstein, Barbara Plank, and Anders Søgaard, *Character-level supervision for low-resource POS tagging*, Proceedings of the Workshop on Deep Learning Approaches for Low-Resource NLP, Melbourne. Association for Computational Linguistics, 2018, 1–11.
- [43] Lyan Verwimp, Joris Pelemans, Hugo Van hamme, and Patrick Wambacq, *Character-word LSTM language models*, Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics:

- Volume 1, Valencia, Spain. Association for Computational Linguistics, 2017, 417–427.
- [44] Quoc Le, Jiquan Ngiam, Adam Coates, Ahbik Lahiri, Bobby Prochnow, and Andrew Ng, *On optimization methods for deep learning*, 28th International Conference on Machine Learning (ICML’11), 2011, 265–272.
 - [45] Alexandre de Brebisson and Pascal Vincent, *An exploration of SoftMax alternatives belonging to the spherical loss family*, International Conference on Learning Representations (ICLR’16).
 - [46] Dhrubajyoti Pathak, Sukumar Nandi, and Priyankoo Sarmah, *AsPOS: Assamese part of speech tagger using deep learning approach*, IEEE/ACS 19th International Conference on Computer Systems and Applications (AICCSA’22), IEEE, 2022.
 - [47] A. K. Barman, J. Sarmah, and S. K. Sarma, *POS tagging of Assamese language and performance analysis of CRF++ and fnTBL approaches*, UKSim 15th International Conference on Computer Modelling and Simulation, 2013, 476–479. DOI: <https://doi.org/10.1109/UKSim.2013.91>
 - [48] Navanath Saharia, Dhrubajyoti Das, Utpal Sharma, and Jugal Kalita, *Part of speech tagger for Assamese text*, ACL-IJCNLP Conference Short Papers (ACLShort’09), Association for Computational Linguistics, 2009, 33–36.
 - [49] https://www.tensorflow.org/api_docs/python/tf/all_symbols
 - [50] <https://keras.io/api/>
 - [51] <https://pandas.pydata.org/docs/>
 - [52] <https://numpy.org/doc/>
 - [53] <https://www.nltk.org/api/nltk.html>
 - [54] <https://scikit-learn.org/0.21/documentation.html>
 - [55] <https://docs.python.org/3/library/collections.html>

[56] <https://flask.palletsprojects.com/en/3.0.x/>

[57] <https://docs.render.com/>